Haskell Notes by Ichi Kanaya

目次

1 Haskell 入門 4
1.1 Haskell について 4
1.1.1 Haskell という森 4
1.1.2 関数型プログラミング 5
1.1.3 Haskell コンパイラの準備 8
1.1.4 余談：本書の構成 9
1.1.5 この章のまとめ 9
1.2 カリー風な書き方 9
1.2.1 変数と関数 9
1.2.2 ラムダ式 11
1.2.3 パタンマッチとガード 12
1.2.4 余談：条件式 13
1.2.5 この章のまとめ 14
1.3 さらにカリー風な書き方 14
1.3.1 演算子 14
1.3.2 関数合成と関数適用 15
1.3.3 高階関数 16
1.3.4 余談：IO サバイバルキット 1 17
1.3.5 この章のまとめ 18
1.4 型 18
1.4.1 データ型 18
1.4.2 カリー化 20
1.4.3 多相型と型クラス 21
1.4.4 余談：全称量化子 23
1.4.5 この章のまとめ 23
1.5 リスト 23
1.5.1 リスト 23
1.5.2 畳み込み 27
1.5.3 マップ 28
1.5.4 余談：IO サバイバルキット 2 29
1.5.5 この章のまとめ 31
1.6 再帰 31
1.6.1 関数の再帰適用 31
1.6.2 リストと再帰 31
1.6.3 末尾再帰 33
1.6.4 余談：遅延評価 34
1.6.5 この章のまとめ 34
1.7 Maybe 35
1.7.1 Possibly 35
1.7.2 Maybe 36
1.7.3 リストと Maybe 39
1.7.4 余談: Either 40
1.7.5 この章のまとめ 40

1.8 関手 40
1.8.1 リストの世界・Maybe の世界 40
1.8.2 関手則 43
1.8.3 アプリカティブ関手 44
1.8.4 余談：関数と関手 46
1.8.5 この章のまとめ 48
1.9 モナド 48
1.9.1 バインド演算子 48
1.9.2 モナド則 49
1.9.3 モノイド 49
1.9.4 余談：IO サバイバルキット 3 51
1.9.5 この章のまとめ 51
1.10 IO 51
1.10.1 IO 51
1.10.2 IO モナド 51
1.10.3 制御構造 51
1.10.4 余談：do 記法 51
1.10.5 この章のまとめ 52
2 Haskell プログラミング 52
3 プログラミングと代数構造 52
3.1 整数 52
3.2 条件式 52
3.3 Y コンビネータ 52
4 Haskell の深い部分 52
4.1 型と種 52
4.1.1 型の計算 52
4.1.2 種 52
4.2 マクロと Template Haskell 52
5 執筆ノート 53

種類	字体・表記法	例
変数・関数	イタリック（1 文字）	x, f
有名な変数・関数・定数	ローマン・小文字	$\mathrm{first}, \mathrm{id}, \mathrm{otherwise}$
文脈に入れる関数（アクション）	ギリシア文字（1 文字）	α, φ
有名なアクション	ボールド・小文字	$\mathbf{main}, \mathbf{print}$
リスト変数	変数名に s をつける	x_{s}
Maybe 変数	変数名に ? をつける	$x_{?}$
Either 変数	変数名に ! をつける	$x_{!}$
一般のコンテナ変数	変数名に $*$ をつける	x_{*}
値コンストラクタ（引数なし）	ローマン（大文字）	$\mathrm{True}, \mathrm{Nothing}$
値コンストラクタ（引数あり）	ローマン（大文字）	Just_x
有名な値コンストラクタ 1	数学記号	$\emptyset, \otimes$
有名な値コンストラクタ 2	特別な括弧で包む	$[x], \langle y \rangle, (\!(u, v)\!)$
型（引数なし）	ボールドイタリック（1 文字）	$\boldsymbol{a}$
型（引数あり）	ボールドイタリック（1 文字）	$\boldsymbol{m_a}$
有名な型（引数なし）	ボールド・大文字	$\mathbf{Int}$
有名な型（引数あり）	特別な括弧で包む	$[\boldsymbol{a}], [\mathbf{Int}]$
ユニット型	括弧	$(\!\mid\!)$
型クラス	サンセリフ	Num
カインド	星，フラクチュール	$\star, \mathfrak{Type}, \mathfrak{K}$
キーワード	固定幅	`let`
無名パラメタ	ひし形（白）	$\lozenge$
無名型パラメタ	ひし形（黒）	$\blacklozenge$

表 1: 凡例 1

種類	字体・表記法	例
集合（数学）	ブラックボード（1 文字）	$\mathbb{Z}$
関手（数学）	スクリプト（1 文字）	$\mathscr{F}$
圏（数学）	カリグラフィ（1 文字）	$\mathcal{C}$

表 2: 凡例 2

1 Haskell 入門

本書はプログラミング言語 Haskell の入門書である．それと同時に，本書はプログラミング言語を用いた代数構造の入門書でもある．プログラミングと代数構造の間には密接な関係があるが，とくに**関数型プログラミング**を実践する時にはその関係を意識する必要が出てくる．本書はその両者を同時に解説することを試みる．

1.1 Haskell について

これからのプログラマにとって Haskell を無視することはできない．Haskell の「欠点をあげつらうことも，攻撃することもできるが，無視することだけはできない」のだ．それは Haskell がプログラミングの本質に深く関わっているからである．

1.1.1 Haskell という森

Haskell というプログラミング言語を知ろうとすると，従来のプログラミング言語の知識が邪魔をする．モダンで，人気があって，Haskell から影響を受けた言語，たとえば Ruby や Swift の知識さえ，Haskell を学ぶ障害になり得る．ではどのようにして Haskell の深みに到達すればいいのだろうか．

その答えは，一見遠回りに見えるが，一度抽象数学の高みに登ることである．

と言っても，あわてる必要はない．

近代的なプログラミング言語を知っていれば，すでにある程度抽象数学に足を踏み入れているからである．そこで，本書では近代的なプログラマを対象に，プログラミング言語を登山口に抽象数学の山を登り，その高みから Haskell という森を見下ろすことにする．

さて，登山口にどのプログラミング言語を選ぶのが適当であろうか．TIOBE Index 2025 年 12 月版によると「ビッグ 5」として Python，C，C++，Java，C# が挙げられている．[1]順位の変動はあるが，他の調査でもビッグ 5 は過去何年も変動していないので，当座は妥当な統計であろう．このうち C は「多くのプログラマが読める」以外にメリットが無く，その唯一のメリットさえ最近は怪しくなっているため，登山口候補から外す．残るは C++，Java，C# グループと Python ということになるが，シンプルであり，かつ Haskell と対極にある言語である Python を登山口に選ぶことにした．

本書では Python コードはこのように登場する．

```python
# Python
print("Hello, world.")
```

本書に示すコードは擬似コードではなく，すべて実行可能な本物のコードである．

[1] https://www.tiobe.com/tiobe-index/

一部の章でどうしても型に触れないといけない部分がある．Python は動的型付け言語であり，型の説明には不適切であるため，この部分だけ理解の助けとして C++によるコードを例示した．この部分はコードを読まなくても先に進める．

ところで，プログラムのソースコードは現代でも ASCII 文字セットの範囲で書くことが標準的である．Unicode を利用したり，まして文字にカラーを指定したり，書体や装飾を指定することは一般的ではない．たとえば変数 `a` のことを a と書いたり $\boldsymbol{a}$ と書いたり $\tilde{a}$ と書いたりして区別することはない．

Haskell プログラマもまた，多くの異なる概念を同じ貧弱な文字セットで表現しなければならない．これは，はじめて Haskell コードを読むときに大きな問題になりえる．たとえば Haskell では `[a]` という表記をよく扱う．この `[a]` は `a` という変数 1 要素からなるリストのこともあるし，`a` 型という仮の型から作ったリスト型の場合もあるが，字面からでは判断できない．もし変数はイタリック体，型はボールドイタリック体と決まっていれば，それぞれ $[a]$ および $[\boldsymbol{a}]$ と区別できたところである．

本書は，異なる性質のものには異なる書体を割り当てるようにしている．ただし，どの表現もいつでも Haskell に翻訳できるように配慮している．実際，本書執筆の最大の困難点は，数学的に妥当で，かつ Haskell の記法とも矛盾しない記法を見つけることであった．

1.1.2 関数型プログラミング

プログラマはなぜ Haskell を習得しなければならないのだろう．それは Haskell と**関数型プログラミング**の間に密接な関係があるからである．

関数型プログラミングとはプログラミングにおける一種のスローガンのようなもので，どの言語を用いたから関数型でどの言語を用いたから関数型ではない，というものではない．しかし，関数型プログラミングを強くサポートする言語と，そうでない言語とがある．ここら辺の事情はオブジェクト指向プログラミングとプログラミング言語の関係と似ている．Haskell は関数型プログラミングを強くサポートし，Python はほとんどサポートしない．

関数型プログラミングの特徴を一言で言えば，プログラム中の**破壊的代入**を禁止することである．変数 x に 1 という数値が一度代入されたら，変数 x の値をプログラム中に書き換える，すなわち破壊的代入をすることはできない．この結果，変数の値はプログラムのどこでも，どの時点で読み出しても同じであることが保証される．これを変数の**参照透過性**と呼ぶ．[2]

プログラム全体に参照透過性があると，そのプログラムはブロックに分割しやすく，各々のブロックは再利用しやすい．またプログラムのどの断片から読み始めても，全体の構造を見失いにくい．これが関数型プログラミングとそれを強くサポートする Haskell を習得する理由である．

参照透過性がもたらすもう一つのボーナスは変数の**遅延評価**である．変数はいつ評価しても値が変わらないのだから，コンパイラは変数をできるだけ遅く評価してよい．この遅延評価によって，Haskell コンパイラは他の言語に見られない**無限リスト**を扱う能力を獲得している．

ここで数学とプログラミングの関係について述べておこう．ある方程式を解くためにコンピュータによって数値シミュレーションを行うとか，非常に複雑な微分を機械的に行うとか，プログラミングによって数学をサポートすることは計算機科学の主たる分野の一つであるが，ここではもっと根源的な話をする．

[2] 変数の値が常に同じなことに加えて，関数に同じ引数を与えた場合に常に同じ結果が得られることを参照透過性と呼ぶ．

数学者もプログラマも**関数**をよく使う．数学者が使う関数とは，引数がいくつかあって，その結果決まる戻り値があるようなものだ．一方でプログラマが使う関数というのは，引数と戻り値はだいたい同じとして，中身に条件分岐があったり，ループがあったり，外部変数を書き換えたり，入出力をしたりする．

どちらも同じ関数であるのに，なぜこうもイメージが違うのだろうか．

もし，我々が関数型プログラミングの原則を忠実に守り，プログラム中のいかなる破壊的代入をも禁止するとすると，両者の関数は全く同じ性格になる．逐次実行も条件分岐もループも，それどころか定数さえ，**ラムダ式**という式だけで書けるようになる．あらゆるプログラムが，最終的には単一のラムダ式で書ける．

ところが，入出力，状態変数，例外など，プログラミングに使われる多くのテクニックは関数の副作用を前提としている．参照透過性と副作用を統一的に扱うためには**モナド**という数学概念が必要である．Haskell はモナドを陽に扱うプログラミング言語である．

♣

関数型プログラミングという考え方自体は，プログラミングに必ずしも束縛されるものではない．

たとえばいま，ある英文テキストファイルから，単語を出現頻度順に取り出したいとしよう．こんなとき，プログラマならば次のように考えるのではないだろうか．

「まずは英文テキスト中の大文字を全部小文字にしよう」「それからアルファベットと空白以外を全部捨ててしまおう」「空白を改行に置き換えると，1 行に 1 単語になるな」「ここまで来たら出現頻度順に並べ直すのはわけないことだ」……

これらの言葉を UNIX の言葉に置き換えてみる．ここでは英文テキストを `the-great-gatsby.txt` としておこう．中身はフィッツジェラルドの名作「The Great Gatsby」（華麗なるギャッツビー）だ．最初の「まずは英文テキスト中の大文字を全部小文字にしよう」は UNIX コマンドの `tr` を使って次のように実現できる．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]'
```

この出力を適当なファイルにリダイレクトすれば，大文字がすべて小文字に置き換わったテキストファイルになる．

次にするのは「それからアルファベットと空白以外を全部捨ててしまおう」だ．これも UNIX コマンド `tr` を使って次のように実現できる．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]'
```

これで単語だけのテキストファイルが出来る．次に空白をすべて改行に置き換えてしまおう．もう一度 `tr` を使う．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]' | tr ' ' '\n'
```

これで 1 行に 1 単語ずつ並べられたテキストファイルがこれで手に入ることになる．空行もあるが，それは問題ない．このファイルの中身をアルファベット順にソートする．それには `sort` コマンドを使う．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]' | tr ' ' '\n'
| sort
```

ソートできれば，同一の連続する行を数え上げて，一つの行にすればいい．それには `uniq` コマンドを `-c` オプション付きで使う．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]' | tr ' ' '\n'
| sort | uniq -c
```

最後に，出現頻度順に逆順ソートをかけると「出現頻度順」になる．それには `sort` コマンドに `-n` と `-r` オプションを付けて以下のようにする．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]' | tr ' ' '\n'
| sort | uniq -c | sort -n -r
```

この出力の先頭 10 行は，そのまま出現頻度上位 10 傑の単語となる．

♣

さて，上述の出現頻度上位単語の抽出のどこが「関数型プログラミング」の考え方だったのだろうか．

実はこんな風に考えることが出来る．入力を x とする．いまの例では x がファイル `the-great-gatsby.txt` である．出力を y とする．ここに y は出現頻度順に並んだ出現頻度と単語のリストである．我々が欲しいのは

$$y = f(x) \tag{1}$$

となるようなプログラム f だった．プログラム f は簡単には手に入らないので，我々は UNIX コマンドの `tr` とか `sort` とか `uniq` とかをつなぎ合わせて作った．それらを「数学風」に書くと次のようになる．

$$\begin{aligned} f_1 &= \mathrm{tr}_{[A\ldots Z]\to[a\ldots z]} \\ f_2 &= \mathrm{tr}_{\overline{[a\ldots z,\square]}\to\emptyset} \\ f_3 &= \mathrm{tr}_{\square\to\swarrow} \\ f_4 &= \mathrm{sort} \\ f_5 &= \mathrm{uniq}_c \\ f_6 &= \mathrm{sort}_{n,r} \end{aligned} \tag{2}$$

そして x から y を得るために

$$y = f_6(f_5(f_4(f_3(f_2(f_1(x)))))) \tag{3}$$

という計算を行った．括弧が多すぎるので，この式を

$$y = f_6 \circ f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1(x) \tag{4}$$

と書き直そう．ここに演算子 $\circ$ は「**関数合成演算子**」だ．

関数合成演算子を使うと，プログラム f を

$$f = f_6 \circ f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1 \tag{5}$$

と定義することも出来る．これが何を意味しているかと言うと，プログラム f を小さな部分プログラム $f_1, f_2, f_3, f_4, f_5, f_6$ に分解したということだ．

分解したなら，合成する方法が必要になる．この例では，プログラムの各部分が他の部分に依存していないために，合成は数学的な関数合成と同じ方法が使える．

上記の例のように，数学における関数合成をそのままプログラミングに持ち込めれば，部分プログラムの合成も見通しが良いものになる．このような考え方が「関数型プログラミング」のエッセンスなのである．

1.1.3 Haskell コンパイラの準備

Haskell はコンパイラ言語であり，またほとんどの OS で標準言語という位置づけでもないため，Haskell コンパイラを自身の OS にインストールする必要がある．Linux, macOS, Windows では Haskell Stack という Haskell 言語環境をインストールするのが簡単である．

macOS では，以下のようにインストールすることを推奨する．

まず Homebrew というパッケージマネージャを macOS にインストールする．それには，コマンドシェルから次のようにする．

```
$ /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Homebrew がインストールできたら，続けて Haskell Stack を次のコマンドでインストールする．

```
$ brew install haskell-stack
```

♣

Windows (Intel)の場合は https://docs.haskellstack.org/en/stable/ から Windows 版インストーラをダウンロードできる．

♣

Haskell Stack をインストールしたら，次のコマンドで新規プロジェクトを作成する．

```
$ stack new myProject
```

自分の関数は `myProject/src/Lib.hs` に書かれている以下のソースコードのうち `someFunc` の定義を書き換えることになる．

```haskell
-- Lib.hs
module Lib
    ( someFunc
    ) where

someFunc :: IO ()
someFunc = putStrLn "someFunc"
```

あるいは `myProject/app/Main.hs` に書かれている `main` の定義を直接書き換えても良い．`Main.hs` は次のようなファイルである．

```haskell
-- Main.hs
```

```haskell
module Main (main) where
import Lib

main :: IO ()
main = someFunc
```

計算機科学の習慣に従って "Hello, World!" と表示するプログラムを書いておこう．それには `Lib.hs` を書き換えて次のようにする．

```haskell
-- Lib.hs
module Lib
    ( someFunc
    ) where

someFunc :: IO ()
someFunc = putStrLn "Hello, World!"
```

`Lib.hs`（または `Main.hs`）を書き換えた後は次のコマンドで実行する．最初のビルド時には，必要な Haskell コンパイラやライブラリがダウンロードされるので少々の時間がかかる．

```
$ stack build
$ stack run
```

1.1.4 余談：本書の構成

📍 **TK** 本書の構成．[3]

1.1.5 この章のまとめ

📍 **TK** この章のまとめ．

1.2 カリー風な書き方

本書では一般の数学書やプログラミングの教科書からは少し異なった記法を用いる．ある概念が発明されてからずっと後になって正しい記法が見つかり，それがきっかけとなって正しく理解されるという現象は歴史上よくあることである．本書でも様々な新しい記号，記法を導入するが，この章では Haskell に近い記法から始めることにする．

数学やプログラミング言語には書き方に一定の決まりがある．この章ではまず「カリー風の」数式記述方式を見てみることにする．「カリー風」というのは，数学者ハスケル・カリーから名前を借りた言い方で，筆者が勝手に命名したものだ．

カリー風の書き方は数学の教科書やプログラミングの教科書で見かけるものとは若干違うが，圧倒的にシンプルで Haskell との親和性も高く，慣れてくると非常に読みやすいものなので，本書でも全面的に採用する．

1.2.1 変数と関数

変数 x に値 1 を代入するには次のようにする．[4]

$$x = 1 \tag{6}$$

[3] 脚注は読まなくて良い．
[4] Haskell では `x = 1` と書く．

変数という呼び名に反して，変数の値は一度代入したら変えられない．そこで変数に値を代入するとは呼ばずに，変数に値を**束縛**するという． 式 6 の右辺のように数式にハードコードされた値を**リテラル**と呼ぶ．

リテラルや変数には**型**がある．型は数学者の**集合**と似た意味で，整数全体の集合 $\mathbb{Z}$ に相当する**整数型**や，実数全体の集合 $\mathbb{R}$ に相当する**浮動小数点型**がある．整数と整数型，実数と浮動小数点型は異なるため，整数型を **Int** で，浮動小数点型を **Double** で表すことにする．[5]

型については節 1.4 で詳しく述べる．

数学者は変数 x が整数であることを $x \in \mathbb{Z}$ と書くが，本書では $x :: \mathbf{Int}$ と書く．これは記号 $\in$ を別の用途に用いるためである．また型の注釈と代入をまとめて $x :: \mathbf{Int} = 1$ と書く事もできる．[6]

本書では変数名を原則 1 文字として，イタリック体で表し w, x, y, z のような n 以降のアルファベットを使う．

♣

変数の値がいつでも変化しないことを**参照透過性**と呼ぶ．プログラマが変数の値を変化させたい，つまり**破壊的代入**を行いたい理由はユーザ入力，ループ，例外，内部状態の変化，大域ジャンプ，継続を扱いたいからであろう．しかし，後に見るようにループ，例外，内部状態の変化，大域ジャンプ，継続に変数の破壊的代入は必要ない．ユーザ入力に関しても章を改めて取り上げる．参照透過性を強くサポートするプログラミング言語をを**関数型プログラミング言語**と呼ぶ．

♣

整数 x に 1 を足す**関数** f は次のように定義できる．

$$fx = x + 1 \tag{7}$$

ここに x は関数 f の**引数**である．引数は括弧でくるまない．[7]

Python や一般的な数学書では引数 x をとる関数 f を $f(x)$ と書くが，括弧は冗長なので今後は fx と書くことにする．[8]

本書では関数名を原則 1 文字として，イタリック体で表し，f, g, h のようにアルファベットの f 以降の文字を使う．ただし有名な関数についてはローマン体で表し，文字数も 2 文字以上とする．たとえば sin などの三角関数や指数関数がそれにあたる．

関数 f に引数 xを「食わせる」ことを**適用**と呼ぶ．もし fx と書いてあったら，それは f と x の積，つまり $f \times x$ ではなく，従来の $f(x)$ すなわち関数 f に引数 x を与えているものと解釈する．高校生向けの数学書でも $\sin x$ のように三角関数に限ってはカリー風に書くことになっているので，まるで馴染みがないということもないだろう．なお，関数はいつも引数の左側に書くことにする．これを「関数 f が変数 x の左から作用する」と言い，また関数 f のことを**左作用素**とも呼ぶ．

[5] Haskell では `Int` および `Double` と書く．

[6] Haskell では $x :: \mathbf{Int}$ を `x :: Int` と書き $x :: \mathbf{Int} = 1$ を `x :: Int = 1` と書く．

[7] Haskell では `f x = x + 1` と書く．

[8] Haskell では関数 `f` に引数 `x` を適用させることを `f x` と書く．数学や物理学では x をパラメタとする関数を $f(x)$ と書く場合もあるし，f のようにパラメタを省略する場合もある．数学や物理学でパラメタを省略した場合は，$f(x_0)$ の意味で $f|_{x=x_0}$ と書くことがある．

変数 x に関数 f を適用して z を得る場合は次のように書く．[9]

$$z = fx \tag{8}$$

♣

複数引数をとる関数を Python や一般的な数学の教科書では $g(x, y)$ と書くが，これも括弧が冗長なので今後は gxy と書く．この場合式 gxy は左を優先して結合する．つまり $gxy = (gx)y$ である．これは引数 y に関数 (gx) が左から作用していると解釈する．関数 (gx) は引数 x に関数 g を作用させて作った関数である．引数に「飢えた」関数 (gx) を**部分適用**された関数と呼ぶ．

関数 f が引数をふたつ取る場合は，次のように書く．[10]

$$z = fxy \tag{9}$$

なお fxy は $(fx)y$ と解釈される．前半の (fx) は 1 引数の関数とみなせる．2 引数関数を連続した 1 引数関数の適用とみなす考え方を，関数の**カリー化**と呼ぶ．

このように式の左側を優先的に演算していくことを**左結合**と呼ぶ．Haskell の場合，関数適用はいつも左結合である．

部分適用の例を見てみよう．例えばふたつの引数のうち大きい方を返す関数 max は $\max xy$ として使われるが，関数適用は左結合であるから $(\max x)y$ としても同じである．そこで $(\max x)$ だけ取り出すと，これは「引数が x よりも小さければ x を，そうでなければ引数を返す関数」とみなすことができる．[11]

1.2.2 ラムダ式

関数の正体は**ラムダ式**である．ラムダ式とは，仮の引数をとり，その値をもとになにがしかの演算を行い，その結果を返す式である．ラムダ式は名前のない関数のようなものだ．それゆえ，無名関数と呼ばれることもある．

例えば引数 x をとり値 $1 + x$ を返すラムダ式を Python では次のように書く．

```
# Python
lambda x: 1 + x
```

一方，我々はより簡潔に

$$\backslash\, x \mapsto 1 + x \tag{10}$$

と書くことにする．

この式は多くの書物で $\lambda x\,.\,1 + x$ と記述されるところである．しかし我々はすべてのギリシア文字を変数名のために予約しておきたいのと，ピリオド記号 (.) が今後登場する二項演算子と紛らわしいため，上述の記法を用いる．[12]

[9] Haskell では `z = f x` と書く．

[10] Haskell では `z = f x y` と書く．

[11] Haskell では $\max xy$ を `max x y` と書く．なお関数 $(\max x)$ のことを $\max_x$ と書く教科書も多い．関数引数を添え字で表す記法は，本書でも後に採用する．

[12] Haskell ではラムダ式 $\backslash\, x \mapsto 1 + x$ を `\x -> 1 + x` と書く．ラムダ式は元々は $\hat{x}\,.\,1 + x$ のように書かれていた．これが次第に $\hat{}x\,.\,1 + x$ となり，$\Lambda x\,.\,1 + x$ そして $\lambda x\,.\,1 + x$ に変化していったと言われている．Haskell が λ の代わりに $\backslash$ 記号を使うのは，その形が似ているからである．

ラムダ式は関数である．ラムダ式を適用するには，ラムダ式を括弧で包む必要がある．例を挙げる．

$$(\backslash\, x \mapsto 1 + x)\ 2 \tag{11}$$

この式は結果として 3 を返す．

複数引数をとるラムダ式は例えば $\backslash\, xy \mapsto x + y$ のように引数を並べて書く．

♣

本書では新たに，次のラムダ式記法も導入する．式中に記号 $\diamond$ が現れた場合，その式全体がラムダ式であるとみなす．記号 $\diamond$ の部分には引数が入る．第 n 番目の $\diamond$ には第 n 番目の引数が入る．例えばラムダ式 $\backslash\, xy \mapsto x + y$ は $\diamond + \diamond$ と書いても良い．式を左から読んで 1 番目の $\diamond$ が元々の x すなわち第 1 引数を，2 番目の $\diamond$ が元々の y すなわち第 2 引数を意味する．この省略記法はプログラミング言語 Scheme における `cut` プロシジャに由来する．[13]

♣

数式が長く続くとき，読みやすさのために**局所変数**を導入すると便利である．例えば

$$y = f(1 + x) \tag{12}$$

という式のうち，先に $1 + x$ を計算して x' のように名前をつけておきたいこともある．そんなときは次のように書く．[14]

$$y = \texttt{let}\ x' \triangleq 1 + x\ \texttt{in}\ fx' \tag{13}$$

なお，局所変数を後ろに回して

$$y = fx'\ \texttt{where}\ x' \triangleq 1 + x \tag{14}$$

と書いても良い．[15]

局所変数はラムダ式を用いたシンタックスシュガーである．式 13 は次の式と等価である．

$$z = (\backslash\, x' \mapsto fx')(1 + x) \tag{15}$$

1.2.3 パタンマッチとガード

関数の定義は，基本的にはラムダ式の変数への代入である．引数 x をとり値 $2 \times x$ を返す関数 f は

$$f = \backslash\, x \mapsto 2 \times x \tag{16}$$

と定義できる．ただし，この省略形として

$$fx = 2 \times x \tag{17}$$

と書いても良い．[16]

[13] Haskell では，中置演算子に限ってこの表現が使える．例えば $(\diamond + \diamond)$ は単に `(+)` と表現できる．ただし Scheme における `cut` プロシジャの `<>` は Haskell にはないため，Scheme でいう `(cut f <> y)` に相当するコードを直接は書けない．

[14] Haskell では `y = let x' = 1 + x in f x'` と書く．

[15] Haskell では `y = f x' where x' = 1 + x` と書く．

[16] Haskell では $f = \backslash\, x \mapsto 2 \times x$ を `f = \ x -> 2 * x` と書き，一方 $fx = 2 \times x$ を `f x = 2 * x` と書く．

関数に**スペシャルバージョン**がある場合はそれらを列挙する．例えば引数が 0 の場合は特別に戻り値が 1 であり，その他の場合は $2 \times x$ を返す関数 f' を考える．このとき f' は以下のように定義できる．これを関数の**パタンマッチ**と呼ぶ．[17]

$$\begin{aligned} f'\ 0 &= 1 \\ f'x &= 2 \times x \end{aligned} \tag{18}$$

関数のパタンマッチは，関数の内部に書いても良い．関数内部にパタンマッチを書きたい場合は次のように書く．

$$f'x = \texttt{case}\ x\ \texttt{of}\ \begin{cases} 0 \dashrightarrow 1 \\ \square \dashrightarrow 2 \times x \end{cases} \tag{19}$$

ここに $\square$ は任意の値の意味である．パタンマッチは上から順番にマッチングしていくため，この場合は 0 以外を意味する．[18]

一部のプログラミング言語では**デフォルト引数**という，引数を省略できるメカニズムがあるが，我々は引数をいつも省略しないことにする．[19]

関数定義にパタンマッチではなく**場合分け**が必要な場合は**ガード**を用いる．例えば引数の値が負の場合は 0 を，0 の場合は 1 を，それ以外の場合は $2 \times x$ を返す関数 f'' は以下のように定義する．[20]

$$\begin{aligned} f''x|_{x<0} &= 0 \\ |_{x\equiv 0} &= 1 \\ |_{\text{otherwise}} &= 2 * x \end{aligned} \tag{20}$$

ガードは上から順にマッチされる．

1.2.4 余談：条件式

関数定義の場合分けを駆使すれば**条件式**はなくても構わないが，条件式の記法があるのは便利である．Python には次のような**制御構造**としての条件文がある．

```python
# Python
def fppp(x):
  if x == 0:
    return 1
  else:
    return sin(x) / x
```

一方，我々は値を持つ**条件式**を考える．我々の条件式とは

$$f'''x = \texttt{if}\ x \equiv 0\ \texttt{then}\ 1\ \texttt{else}\ {}^{\sin x}\!/_{x} \tag{21}$$

のように `if` 節，`then` 節，及び `else` 節からなるものであって，`then` 節も `else` 節も省略できないものとする．`if` 節の式の値が真 (True) であれば `then` 節の式が評価され，偽 (False) であれば `else` 節の式が評価される．我々の条件式は C における条件演算子（三項演算子）と等し

[17] Haskell では `f' 0 = 1; f' x = 2 * x` と書く．
[18] Haskell では `f' x = case x of { 0 -> 1; _ -> 2 * x }` と書く．
[19] Haskell にもデフォルト引数はない．
[20] Haskell では `f'' x | x < 0 = 0  | x == 0 = 1  | otherwise = 2 * x` と書く．もっとも，この記法を使う場合は改行を適度に入れたほうが読みやすい．

く見えるが，Haskell の場合は遅延評価が行われるため，結果として条件式の**短絡評価**が行われる点が異なる．[21]

1.2.5 この章のまとめ

📍 **TK** この章のまとめ．

1.3 さらにカリー風な書き方

我々は関数とラムダ式の「カリー風」な書き方を見てきた．この章ではさらに演算子，関数合成についても「カリー風」な書き方を見ていく．

1.3.1 演算子

演算子は関数の特別な姿である．演算子は**作用素**と呼んでも良い．どちらも英語の operator の和訳である．演算子は普通アルファベット以外のシンボル 1 個で表現し，変数や関数の前に置いて直後の変数や関数に作用させるか，2 個の変数や関数の中間に置いてその両者に作用させる．例えば $-x$ のマイナス記号 $(-)$ は変数の前に置いて直後の変数 x に作用する演算子であり，$x+y$ のプラス記号 $(+)$ は 2 個の変数の間に置いてその両者 (x, y) に作用する．

1 個の変数または関数に作用する演算子を**単項演算子**と呼び，2 個の変数または関数に作用する演算子を**二項演算子**と呼ぶ．本書では単項演算子はすべて変数の前に置く，すなわち**前置**する．前置する演算子のことを**前置演算子**と呼ぶが，数学者は同じものを左作用素と呼ぶ．

Haskell には単項マイナス $(-)$ を除いて他に単項演算子はない．今後，誤解を避けるために数式中の $-x$ はすべて $(-x)$ と書く．

二項演算子のうちよく使われるものは和 $(+)$，積 $(\times)$，論理和 $(\vee)$，論理積 $(\wedge)$，同値 $(\equiv)$，大なり $(>)$，小なり $(<)$ 等である．二項演算子はたとえ積記号であっても省略できない．二項演算子は多数あるので，その都度説明する．[22]

二項演算子は**中置**することが基本であるが，括弧で包むことで前置することも可能である．任意の二項演算子 $\circ$ について $x \circ y$ 及び $(\circ)xy$ は全く同じ意味である．すなわち

$$(\circ)xy = x \circ y \tag{22}$$

である．従って，二項演算子と 2 引数関数に本質的な差はない．本書では演算子と関数という用語は全く同じ意味で用いる．[23]

一般の関数が左結合であることを思い出すと，二項演算子を関数に見立てた $(\circ)$ についても

$$(\circ)xy = ((\circ)x)y \tag{23}$$

であるから，部分適用が可能である．この式から第 2 引数 y を取り除いて $((\circ)x)$ という「餓えた」1 引数関数を取り出せる．例えば関数 $((+)1)$ は引数に 1 を加える関数である．[24]

前置される二項演算子 $(\circ)$ は，ラムダ式 $(\diamond \circ \diamond)$ の無名パラメタ $\diamond$ を省略したものと考えても良い．また $(\diamond \circ x)$ や $(x \circ \diamond)$ から無名パラメタを省略した $(\circ\, x)$ と $(x\, \circ)$ も有効な表現であり，特別に**セクション**と呼ばれる．

二項演算子 $\circ$ に対して $(\circ\, x)$ および $(x\, \circ)$ はそれぞれ以下の通りである．

---

[21] Haskell では $fx = \texttt{if}\ x \equiv 0\ \texttt{then}\ 0\ \texttt{else}\ {}^{(\sin x)}\!/_{x}$ を `f x = if x == 0 then 1 else (sin x) / x` と書く．

[22] Haskell では $\wedge$ を `&&` と書き，$\vee$ を `||` と書く．

[23] Haskell では任意の二項演算子を括弧で包むことで前置演算子として使うことができる．例えば `x + y` と `(+) x y` は同じ結果を返す．

[24] Haskell では $((+)1)$ を `((+)1)` と書く．

$$
\begin{aligned}
(\circlearrowleft x) &= (\lozenge \circlearrowleft x) \\
(x \circlearrowleft) &= (x \circlearrowleft \lozenge) = ((\circlearrowleft)x)
\end{aligned}
\tag{24}
$$

例えば $(1+)$ は $((+)1)$ と等価であり，これは $(+1)$ とも等価である．ただし，マイナス演算子 $(-)$ だけは例外で，(-1) はマイナス 1 を表す．負の数をいつも括弧で包んでおくのは良いアイディアである．[25]

なお，二項演算子の結合性，すなわち左結合か右結合かは，演算子によって異なる．また演算の優先順位を明示的に与えるために括弧が用いられる．

一般の関数 f を中置演算子に変換する記号 ‘ f ’ を今後用いる．この記号を用いると値 fxy のことを x ‘ f ’ y と書くことができる．[26]

1.3.2 関数合成と関数適用

ある変数に複数の関数を順に適用することはよくあることである．例えば次のようにしたいことがある．

```python
# Python
y = f(x)
z = g(y)
```

あるいは同じことであるが次のようにしたいことがある．

```python
# Python
z = g(f(x))
```

このコードを本書の記法で書けば $z = g(fx)$ である．この式から括弧を省略して $z = gfx$ としてしまうと，関数適用は左結合するから $z = (gf)x$ の意味になってしまう．関数 g が引数に関数を取るので無い限り (gf) は無意味なので，$z = g(fx)$ の括弧は省略できない．

ここで，引数のことは忘れて，関数 f と関数 g を先に**合成**しておきたいとしよう．その合成を $g \circ f$ と書く．演算子 $\circ$ は**関数合成演算子**と呼ぶ．合成はラムダ式を使って $g \circ f = g(f\lozenge)$ と定義できる．関数合成演算子 $\circ$ は関数適用よりも優先順位が高く，$(g \circ f)x$ は単に $g \circ fx$ と書いても良い．この記法は括弧の数を減らすためにしばしば用いられる．[27]

関数合成演算子は，連続して用いることができる．関数合成演算子は左結合するので，関数 f, g, h について $h \circ g \circ f = (h \circ g) \circ f$ であるが，これを展開すると以下のようになる．

$$
\begin{aligned}
(h \circ g) \circ f &= (h \circ g)(f\lozenge) \\
&= h(g(f\lozenge)) \\
&= h \circ (g \circ f)
\end{aligned}
\tag{25}
$$

そのため $h \circ g \circ f = h \circ (g \circ f)$ である．つまり，関数合成は順序に依存しない．

♣

📍 **TK** 関数合成則

[25] Haskell は $(1+)$ を `(1+)` と書く．また `(-1)` はセクションではなくマイナス 1 を表す（`-1` というリテラルとみなされる）．ただし `(- 1)` のように空白を挟んでも同じくマイナス 1 とみなされる（`1` というリテラルに単項マイナス演算子が適用される）．

[26] Haskell では x ‘ f ’ y を `x` ` `f` ` `y` と書く．

[27] Haskell では関数 `g` と関数 `f` の合成は `g.f` である．式 $z = g \circ fx$ は `z = g.f x` と書く．

恒等関数の存在

任意の関数 f に対して恒等関数 id ただし $\mathrm{id} \circ f = f$ が存在する．

結合則

任意の関数 f, g, h について $(h \circ g) \circ f = h \circ (g \circ f)$ が成り立つ．

♣

関数合成演算子とは逆に，結合の優先順位の低い**関数適用演算子**も考えておくと便利なこともある．関数適用演算子 $ を次のように定義しておく．

$$f \,\$\, x = fx \tag{26}$$

演算子 $ の優先順位は関数適用も含めあらゆる演算子よりも低いものとする．関数適用演算子を用いて $z = g(fx)$ を書き直すと $z = g \,\$\, fx$ となる．演算子 $ の優先順位は足し算よりも低いので $f(x+1)$ は $f \,\$\, x + 1$ と書くこともできる．演算子 $ を閉じ括弧のいらない開き括弧と考えてもよい．[28]

関数適用演算子のもう一つの興味深い使い方は，関数適用演算子の部分適用である．セクション $(\diamond \,\$\, x)$ を用いると $(\diamond \,\$\, x)f = f \,\$\, x$ であるから，関数適用演算子を用いて引数を関数に渡すことができる．[29]

1.3.3 高階関数

関数を引数に取ったり，あるいは関数を返す関数のことを**高階関数**と呼ぶことがある．関数合成演算子と関数適用演算子は高階関数の好例である．

他に例えば，引数として整数 a を取り，関数 $fx = a + x$ を返すような関数 g を次のように定義することが出来る．

$$ga = a + \diamond \tag{27}$$

このとき，

$$\begin{aligned} f &= g\ 100 \\ x &= f\ 1 \end{aligned} \tag{28}$$

とすれば $x = 101$ を得る．[30]

高階関数は今後度々顔をだすことになる．後で登場する**マップ演算子**や**畳込み演算子**は高階関数の一種である．

♣

ラムダ式をサポートするほとんどのプログラミング言語は，**レキシカルクロージャ**をサポートする．レキシカルクロージャとは，ラムダ式が定義された時点での，周囲の環境をラムダ式に埋め込む機構である．例えば

$$\begin{aligned} a &= 100 \\ f &= a + \diamond \end{aligned} \tag{29}$$

[28] Haskell では $g \,\$\, fx$ を `g $ f x` と書く．
[29] Haskell では $(\diamond \,\$\, x)f$ を `($x)f` と書く．
[30] Haskell では $ga = a + \diamond$ を $ga = \backslash x \mapsto a + x$ と展開しておいて `g a = \ x -> a + x` と書く．

というラムダ式があるとする．当然我々は関数 f がいつも $f = 100 + \diamond$ であることを期待するし，Haskell においてはいつも保証される．[31]

ところが，参照透過性のない言語，言い換えると変数への破壊的代入が許されている言語では，変数 a の値がいつ変わっても不思議ではない．そこで，それらの言語では関数 f が定義された時点での a の値を，関数 f の定義に含めておく．これがレキシカルクロージャの考え方である．

Haskell ではそもそも変数への破壊的代入がないので，関数 f がレキシカルクロージャであるかどうか悩む必要はない．あえて言えば，Haskell ではラムダ式はいつもレキシカルクロージャである．もしあなたのそばの C++プログラマが「え？　Haskell にはレキシカルクロージャが無いの？」などと聞いてきたら，「ええ，Haskell には破壊的代入すらありませんから」と答えておこう．

1.3.4 余談：IO サバイバルキット 1

ここで，実用的な Haskell プログラムについて触れておきたいと思う．

プログラムとは合成された関数である．多くのプログラミング言語では，プログラムそのものに main という名前をつける．本書では **IO モナド**の章で述べる理由によって，main 関数をボールド体で **main** と書く．

実用的なプログラムはユーザからの入力を受け取り，関数を適用し，ユーザへ出力する．Haskell ではユーザからの 1 行の入力を **getLine** で受け取り，変数の値を **print** で書き出せる．ここに **getLine** と **print** は関数（ファンクション）ではあるが，特別に**アクション**とも呼ぶ．関数 **main** もアクションである．

ユーザ入力をただ受け取り，そのままユーザへ出力するプログラムを Haskell で書くと次のようになる．[32]

$$\mathbf{main} = \mathbf{print} \heartsuit \mathbf{getLine} \tag{30}$$

新しい演算子 ♡ は新たな関数合成演算子で，アクションとアクションを合成するための特別な演算子である．詳細は**モナド**の章で述べる．

♣

何らかの数値計算を行うプログラムは，ユーザ入力を数値として読み取り，数値に作用する関数を適用し，結果をユーザへ出力する．つまり 式 30 の **print** と **getLine** の間に数値計算を行う任意の関数 f を挿入すれば良いことになる．ただし関数 f は数値を受け取って数値を返すものだから，ユーザ入力すなわち**文字列**を数値に変換する必要がある．幸い Haskell は型の違いを吸収する read 関数を提供している．

いま，任意の関数として引数 x の 2 乗を求める関数 f を次のように定義しよう．

$$\begin{aligned} f &:: \mathbf{Double} \to \mathbf{Double} \\ fx &= x \times x \end{aligned} \tag{31}$$

式 31 の 1 行目は関数 f の**型**を表している．型に関しては後述する．

ユーザからの入力に関数 f を適用してユーザへ出力するプログラムを Haskell で書くと次のようになる．

[31] Haskell では `a = 100; f = \x -> a + x` と書く．
[32] Haskell では `main = print =<< getLine` と書く．

$$\mathbf{main} = \mathbf{print} \circ f \circ \mathrm{read} \heartsuit \mathbf{getLine} \tag{32}$$

式 31 と式 32 に対応する Haskell コードは次のようになる．

```haskell
-- Haskell
f :: Double -> Double
f x = x * x
main = print . f . read =<< getLine
```

Haskell Stack を使う場合は `Main.hs` を次のように書き換えると良いだろう．

```haskell
-- Main.hs
module Main (main) where
f :: Double -> Double
f x = x * x
main :: IO ()
main = print . f . read =<< getLine
```

我々はひとまずこれで，最低限の入出力と数値計算を行うプログラムが書けたわけである．いま，以下のような数値だけを記述したファイル `input.txt` があるとしよう．

```
4
```

ここで，シェルで次のように実行する．

```
$ stack run < input.txt
```

すると，次のような出力が得られる．

```
16.0
```

このように，Haskell ではプログラムの実行によってユーザ入力を受け取り，ユーザへ出力を行うことができる．

いまはまだ「ベイビー・プログラム」であるが，今後はより実用度を増していく．

1.3.5 この章のまとめ

📍 **TK** この章のまとめ．

1.4 型

Haskell の変数，関数にはすべて型がある．プログラマの言う型とは，数学者の言う集合のことである．本章では，Haskell が扱う基本的な型であるデータ型と，パラメトリックな型である多相型，および型の型である型クラスについて述べる．また関数のカリー化についても述べる．

1.4.1 データ型

Haskell のすべての変数，関数には**型**がある．代表的な型には整数型，浮動小数点型，ブール型，文字型がある．整数型を **Int** で，浮動小数点型を **Double** で表すことはすでに述べたとおりである．

型とは変数が取りうる値に言語処理系が与えた制約のことである．Haskell を含む多くのコンパイラ言語は**静的型付け**と言って，コンパイル時までに変数の型が決まっていることを

プログラマに要求する．一方，Python のようなインタプリタ言語はたいてい**動的型付け**と言って，プログラムの実行時まで変数の型を決めない．

変数に型の制約を設ける理由は，プログラム上のエラーが減ることを期待するためである．例えば真理値が必要とされるところに整数値の変数が来ることは悪い予兆である．一方で C17 までの C 言語のように全ての変数にいちいち型を明記していくのも骨が折れる．

数学者や物理学者は変数に型の制約を求める一方，新しい変数の型は明記せず読者に推論させる方法をしばしばとる．例えば物理学では，質量 m は「スカラー」という型を持つし，速度 v は「3 次元ベクトル」という型を持つ．スカラーと 3 次元ベクトルの間に足し算は定義されていないため，例えば $m + v$ という表記を見たときに，両者の型を知っていれば直ちにエラーであることがわかる．

Haskell はコンパイラが型推論を行うことで，型が自明の場合は型を省略することができる．[33]

Haskell にはよく使う型が予め用意されている．

Haskell には 2 種類の整数型があり，ひとつは**固定長整数型**で，もうひとつは**多倍長整数型**である．Haskell では前者を `Int` で，後者を `Integer` で表す．多倍長整数型はメモリの許す限り巨大な整数を扱えるので，整数全体の集合に近いのであるが，本書では主に固定長整数型を用いる．この **Int** は C の `int` と似た「計算機にとって都合の良い整数」である．計算機にとって都合の良い整数とは，例えば 64 ビット計算機の場合 -2^{63} から $2^{63} - 1$ の間の整数という意味である．[34]

変数 x の型が **Int**のとき，以下のように**型注釈**を書く．[35]

$$x :: \mathbf{Int} \tag{33}$$

計算機は残念ながら無限精度の実数を扱えない．そこで標準精度（単精度）の**浮動小数点数型**である **Float** と，**倍精度浮動小数点型**である **Double** が提供される．[36]

論理型は論理値 True または False のいずれかしか値をとれない型である．論理型のことを **Bool**と書く．[37]

もう一つ，計算機ならではの型がある．それは **Int** とよく似ているが，特別に文字を扱うために考えられた**文字型** **Char** である．文字といってもその中身は整数である．整数ではあるが，わざわざ別な型とするのには理由がある．[38]

理由の第一は，文字が小さな整数であるため，文字型を独立して定義しておくことでメモリを節約できるのである．特にメモリが高価であった時代はこれが唯一の理由であった．現在でも，整数が一般に 64 ビットを消費するのに対し，UTF-8 文字エンコードを用いている場合，アルファベットは 8 ビットしか消費しない．

理由の第二は，単純に整数と文字が異なるからである．文字を表す変数に整数を代入するのは悪い兆しである．

理由の第三は，文字が数値にエンコードされる方式が可変長である場合に備えて，整数と区別しておくためである．例えば UTF-8 文字エンコードは可変長エンコーディングを行う．

---

[33] C23 および C++11 以降は `auto` キーワードを導入し，Haskell と同様の型推論を行うようになった．
[34] Haskell では **Int** を `Int` で表す．
[35] Haskell では `x :: Int` と書く．
[36] Haskell では **Float**, **Double** をそれぞれ `Float` と `Double` で表す．
[37] Haskell ではブール型を `Bool` と書く．
[38] Haskell では **Char** を `Char` で表す．

これらのような基本的な型を**データ型**と呼ぶ.

♣

関数にも型がある．例えば整数引数を一つ取り，整数を返す関数 f は

$$f :: \mathbf{Int} \to \mathbf{Int} \tag{34}$$

という型を持つ．この型は

$$f :: \underbrace{\mathbf{Int}}_{x} \to \underbrace{\mathbf{Int}}_{fx} \tag{35}$$

のようにイメージすると良い. これは関数 f が集合 **Int** から集合 **Int** への**写像**であると読む.[39]

2 引数関数の型は次のように注釈できる．[40]

$$f :: \mathbf{Int} \to \mathbf{Int} \to \mathbf{Int} \tag{36}$$

ここで関数 f は整数型の引数をふたつとり，整数型の値を返す．型 $\mathbf{Int} \to \mathbf{Int} \to \mathbf{Int}$ は $\mathbf{Int} \to (\mathbf{Int} \to \mathbf{Int})$ と解釈される.

$(\mathbf{Int} \to \mathbf{Int})$ 型の関数を受け取り $(\mathbf{Int} \to \mathbf{Int})$ 型の関数を返す関数は次の型を持つ．[41]

$$f :: (\mathbf{Int} \to \mathbf{Int}) \to (\mathbf{Int} \to \mathbf{Int}) \tag{37}$$

なお後半の括弧は省略可能なので次のように書いても良い.

$$f :: (\mathbf{Int} \to \mathbf{Int}) \to \mathbf{Int} \to \mathbf{Int} \tag{38}$$

Haskell ではすべての変数, 関数に型があり, 型はコンパイル時に決定されていなければならない．ただし，式から**型推論**が行える場合は型注釈を省略できる.

1.4.2 カリー化

Haskell では，どのような関数であれ引数は 1 個しかとらない．引数が 2 個あるように見える関数として，例えば gxy があったとしよう．ここに g は関数，x と y は変数である．関数適用は左結合であるから，これは $(gx)y$ である．ここに (gx) は引数 y をとる関数であると見ることができる．つまり，関数 g とは引数 x をとり「引数 y をとって値を返す関数 (gx) を返す」関数であると言える.

二項演算 $x + y$ は $(+)xy$ とも書けたことを思い出そう. これも左結合を思い出すと $(+)xy = ((+)x)y$ であるから，y という引数を $((+)x)$ という関数に食わせていると解釈できる.

ラムダ式の場合は話はもっと単純で，形式的に

$$\backslash\, xy \mapsto x + y = \backslash\, x \mapsto (\backslash\, y \mapsto x + y) \tag{39}$$

のように展開すれば 1 引数にできる．矢印 $\mapsto$ は**右結合**である．そこでこのラムダ式は括弧を省略して

$$\backslash\, xy \mapsto x + y = \backslash\, x \mapsto \backslash\, y \mapsto x + y \tag{40}$$

とも書かれる.

---

[39] Haskell では `f :: Int -> Int` と書く．なお正確には矢印記号 $(\to)$ は型コンストラクタである．無名の型引数を $\blacklozenge$ で表すと $\to$ は $\langle \blacklozenge \to \blacklozenge \rangle$ という，ふたつの型引数を取る型コンストラクタである.

[40] Haskell では `f :: Int -> Int -> Int` と書く.

[41] Haskell では `f :: (Int -> Int) -> (Int -> Int)` と書く.

複数引数をとる関数を 1 引数関数に分解することを**カリー化**と呼ぶ．これはこの分野の先駆者であるハスケル・カリーの名前に由来する．

整数引数を二つ取り，整数を返す関数 g は

$$g :: \mathbf{Int} \to \mathbf{Int} \to \mathbf{Int} \tag{41}$$

という型を持つ．写像の矢印記号は右結合するので，これは

$$g :: \mathbf{Int} \to (\mathbf{Int} \to \mathbf{Int}) \tag{42}$$

と同じ意味である．上式は

$$g :: \underbrace{\mathbf{Int}}_{x} \to \underbrace{\overbrace{\mathbf{Int}}^{y} \to \overbrace{\mathbf{Int}}^{(gx)y}}_{gx} \tag{43}$$

のようにイメージすると良い．

自然言語で考えると **Int** 型の引数を一つ取り，**Int** 型の引数を一つ取って **Int** 型の値を返す関数を返す，と読める．[42]

♣

Haskell には**タプル**という型がある．タプルとは，複数の変数を組み合わせたもので，例えば変数 x と y をひとまとめにした $(\!|x, y|\!)$ はタプルである．変数 x, y の型は同じでも良いし，異なっても良い．[43]

タプルの型は，要素の型をタプルにしたものである．例えば **Int** が 2 個からなるタプルの型は次のようになる．[44]

$$z :: (\!|\mathbf{Int}, \mathbf{Int}|\!) \tag{44}$$

いまタプルを引数に取る関数 $f(\!|x, y|\!) = x + y$ があったとしよう．Haskell にはタプルをとる関数をカリー化する関数 curry があり，$(\mathrm{curry}\ f)xy$ は $x + y$ になる．

逆に，カリー化された関数 $f'xy = x + y$ に関しては $(\mathrm{uncurry}\ f')(\!|x, y|\!)$ のように**アンカリー化**することで，タプルに適用することができる．

タプルの中身の個数は 0 個または 2 個以上でなければならず，上限は処理系によって定められている．2 個の変数からなるタプルを特別にペア，3 個の変数からなるタプルを特別にトリプルと呼ぶ．中身が 0 個のタプルを $\emptyset$ で表し，特別に**ユニット**と呼ぶ．[45]

ユニット $\emptyset$ の型は**ユニット型**で，変数 x がユニット型の場合は次のように書く．[46]

$$x :: (\!|\,|\!) \tag{45}$$

1.4.3 多相型と型クラス

整数型 (**Int**) と浮動小数点型 (**Float**) はよく似ている．どちらも値同士を比較可能で，それ故どちらにも等値演算子が定義されている．

---

[42] 関数の型に出てくる $\to$ は 2 引数をとる型コンストラクタである．型コンストラクタに関しては〜〜〜で詳しく述べる．例えば $\boldsymbol{a} \to \boldsymbol{b}$ という型は $(\to)\boldsymbol{ab}$ の別名であり，型コンストラクタ $(\to)$ に引数 $\boldsymbol{a}$ と $\boldsymbol{b}$ を与えたものと読む．

[43] Haskell では $(\!|x, y|\!)$ を `(x, y)` と書く．

[44] Haskell では `z :: (Int, Int)` と書く．

[45] GHC v8.2.1 は最大 62 個の変数からなるタプルまで生成できる．

[46] Haskell では `x :: ()` と書く．

整数型の等値演算子は

$$(\equiv) :: \mathbf{Int} \to \mathbf{Int} \to \mathbf{Bool} \tag{46}$$

であり，浮動小数点型の等値演算子は

$$(\equiv) :: \mathbf{Double} \to \mathbf{Double} \to \mathbf{Bool} \tag{47}$$

である．

このように型が異なっても（だいたい）同じ意味で定義されている演算子のことを**多相的**な演算子と呼ぶ．等値演算子は多相的な演算子の例である．

具体的な型を指定せずに，仮の変数で表したものを**型パラメタ**と呼ぶ．我々は型パラメタをボールドイタリック体で表す．いま型を表す仮の変数を $\boldsymbol{a}$ として，等値演算子の型を次のように表現してみよう．

$$(\equiv) :: \boldsymbol{a} \to \boldsymbol{a} \to \mathbf{Bool} \tag{48}$$

このような型パラメタを用いた型を総称して**多相型**と呼ぶ．

実は式 48 は不完全なものである．このままでは型 $\boldsymbol{a}$ に何の制約もないため，等値演算の定義されていない型が来るかもしれないからである．そこで，型自身が所属する，より大きな型があるとしよう．そのような型を我々は**型クラス**と呼ぶ．例えば型 $\mathbf{Bool}, \mathbf{Int}, \mathbf{Integer}, \mathbf{Float}, \mathbf{Double}$ は全て等値演算が定義できるので，型クラス Eq に属すとする．この関係を我々は

$$\mathsf{Eq} \supset \mathbf{Bool}, \mathbf{Int}, \mathbf{Integer}, \mathbf{Float}, \mathbf{Double} \tag{49}$$

と書く．ここに $\mathsf{Eq} \supset \boldsymbol{a}$ と書いて「型 $\boldsymbol{a}$ は型クラス Eq の**インスタンス**である」と読む．

式 48 に型クラスの制約を加えてみよう．型 $\boldsymbol{a}$ は型クラス Eq に属さなければならないから，新たな記号 $\Rightarrow$ を使って

$$(\equiv) :: \mathsf{Eq} \supset \boldsymbol{a} \Rightarrow \boldsymbol{a} \to \boldsymbol{a} \to \mathbf{Bool} \tag{50}$$

と書くことにする．[47]

型 $\boldsymbol{a}$ の変数同士の間で大小関係が定義されている場合，かつその型が型クラス Eq に属する場合，その型は型クラス Ord にも属する．型クラス Ord に属する型は比較演算子 $<, \leq, \geq, >$ を提供する．例えば型 **Int** は型クラス Ord に属すが，型 **Bool** は型クラス Ord に属さない．

型 $\boldsymbol{a}$ の変数同士の間で四則演算関係が定義されている場合，かつその型が型クラス Eq に属する場合，その型は型クラス Num にも属する．型クラス Num に属する型は二項演算子 $+, -, *, /$ を提供する．ここに $-$ は二項演算子のマイナスである．

型 $\boldsymbol{a}$ が型クラス Ord 及び型クラスNum に属しているとき，かつそのときに限り，型 $\boldsymbol{a}$は型クラス Real にも属する．

型 $\boldsymbol{a}$ の変数について，一つ小さい値を返す関数 pred と一つ大きい値を返す関数 succ が定義されているとき，かつそのときに限り，型 $\boldsymbol{a}$ は型クラス Enum に属する．

型 $\boldsymbol{a}$ が型クラス Real 及び型クラスEnum に属しているとき，かつそのときに限り，型 $\boldsymbol{a}$ は型クラス Integral にも属する．

[47] Haskell では `(==) :: Eq a => a -> a -> Bool` と書く．記号 $\supset$ は省略する．

便利な型変換演算子をひとつ紹介しておこう．型変換演関数 fromIntegral は

$$\text{fromIntegral} :: \mathsf{Integral} \supset \boldsymbol{a} \Rightarrow \boldsymbol{a} \to \boldsymbol{b} \tag{51}$$

という型を持ち，Integral 型クラスの型の変数を任意の型へ変換する．例えば，

$$x :: \mathbf{Double} = \text{fromIntegral } 1 :: \mathbf{Int} \tag{52}$$

とすることで，**Double** 型の変数 x に **Int** 型の定数を代入できる．[48]

1.4.4 余談：全称量化子

型 $\boldsymbol{a}$ の変数を引数に取り，型 $\boldsymbol{a}$ の戻り値を返す関数 f の型注釈は

$$f :: \boldsymbol{a} \to \boldsymbol{a} \tag{53}$$

であった．この記法は実はシンタックスシュガーで，本来は次のように書くべきものである．[49]

$$f :: \forall \boldsymbol{a} \mapsto \boldsymbol{a} \to \boldsymbol{a} \tag{54}$$

ここに $\forall$ は**全称量化子**という記号で，型の世界でのラムダ (\) に相当する．

📍 **TK** 型に対する演算

1.4.5 この章のまとめ

📍 **TK** この章のまとめ

1.5 リスト

型から作る型を**コンテナ型**と呼ぶ．代表的なコンテナはある型のホモジニアスな配列であるリストである．この章ではリストと，リストに対する重要な演算である畳み込み，マップを取り扱う．

1.5.1 リスト

同じ型の値を一列に並べたもの，つまり**ホモジニアスな配列**のことを**リスト**と呼ぶ．Python ではリスト xs を次のように定義できる．

```python
# Python
xs = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

我々も 0 から始まり 9 まで続く整数のリストを $[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]$ と書くことにしよう．ただし，これでは冗長なので**等差数列**に限って簡略化した書き方を許す．例えば 0 から 9 までのリストは $[0, 1, \ldots, 9]$ と書いても良い．[50]

リストの中身の一つ一つの値のことを**要素**と呼ぶ．要素のことは元と呼んでも良いが，本書では要素と呼ぶことにする．要素も元も英語の element の和訳である．

複数の型の要素が混在してもよい配列のことを**ヘテロジニアスな配列**と呼び，ホモジニアスな配列とは区別する．

[48] Haskell では `x :: Double = fromIntegral 1 :: Int` と書く．
[49] Haskell では `f :: forall a . a -> a` と書く．記号 $\forall$ が `forall` であり，記号 $\mapsto$ が `.` である．
[50] Haskell では `[0, 1..9]` と書く．ピリオドの数に注意しよう．

今後，リストを指す変数は，リストであることを忘れないように変数名に s をつけて

$$x_{\mathrm{s}} = [0, 1, ..., 9] \tag{55}$$

と書くことにする．なお，変数 x とリスト変数 x_{s} は異なる変数であるとする．[51]

♣

リストは空でもよい．**空リスト**は $\emptyset$ で表す．[52]

関数 null はリストが空リストかどうかを判定する．リスト x_{s} が空リストの場合 null x_{s} は True を，そうでなければ False を返す．関数 null は

$$\mathrm{null} :: [\boldsymbol{a}] \to \mathbf{Bool} \tag{56}$$

である．

♣

Python では**リスト内包表記**が使える．例えば 0 から 9 までの倍数のリストは次のように作った．

```
# Python
xs = [x * 2 for x in range(0, 10)]
```

ここに `range(a, b)` は `a` から増加する方向に連続する `b` 個の整数からなるリストを返す Python の関数である．

我々も内包表記を

$$x_{\mathrm{s}} = [x \times 2 \mid x \in [0, 1, ..., 9]] \tag{57}$$

のように書こう．ここに右辺のリストから一つずつ要素を取り出して左辺に代入する演算子 $\in$ を用いた．[53]

内包表記の式は複数あっても良い．例えば

$$x_{\mathrm{s}} = [x + y \mid x \in [0, 1, ..., 9], y \in [0, 1, ..., 5], x + y > 3] \tag{58}$$

は $0 \leq x \leq 9$ かつ $0 \leq y \leq 5$ の範囲で $x + y > 3$ となる x 及び y から $x + y$ を並べたリストである．[54]

同じことを Python で書くと次のようになる．

```
# Python
xs = [x + y for x in range(0, 10) for y in range(0, 6) if x + y > 3]
```

♣

整数型 (**Int**) のリストは [**Int**] と書き，整数のリスト型と呼ぶ．一般に $\boldsymbol{a}$ 型のリストを $[\boldsymbol{a}]$ と書く．仮の型である $\boldsymbol{a}$ の事を**型パラメタ**と呼ぶ．

[51] Haskell では s を変数名にくっつけて `xs = [0, 1..9]` のように書く習慣がある．なおピリオドの数に注意しよう．

[52] Haskell では空リストを `[]` で表す．

[53] Haskell では $x_{\mathrm{s}} = [x \times 2 \mid x \in [0, 1, ..., 9]]$ を `xs = [x * 2 | x <- [0, 1..9]]` と書く．

[54] Haskell では `xs = [x + y | x <- [0, 1..9], y <- [0, 1..5], x + y > 3]` と書く．

型 $\boldsymbol{a}$ から型 $[\boldsymbol{a}]$ を生成する演算子を**リスト型コンストラクタ**と呼んで [] と書く．型 $[\boldsymbol{a}]$ は $[]_{\boldsymbol{a}}$ のシンタックスシュガーである．[55]

型コンストラクタの概念は Python には無い（必要無い）が，静的型付け言語である C++の クラステンプレートが相当する．

♣

$[x]$ のように $\boldsymbol{a}$ 型の変数 x を入れた $[\boldsymbol{a}]$ 型の変数を作る演算子を**リスト値コンストラクタ**と呼ぶ．$[\boldsymbol{a}]$ 型の変数のことを**リスト変数**とも呼ぶ．$\boldsymbol{a}$ 型の変数 x からリスト値コンストラクタを使ってリスト x_{s} を作ることは

$$x_{\mathrm{s}} = [x] \tag{59}$$

と書く．[56]

リスト型を表す $[\boldsymbol{a}]$ と，1 要素のリストである $[x]$ の違いにはいつも気をつけておこう．本書では中身がボールド体ならばリスト型，中身がイタリック体ならリスト値である．

ある型を包み込んだ別の型を一般にコンテナと呼ぶのであった．コンテナ型は多相型の一種である．また，コンテナ型の変数を**コンテナ変数**と呼ぶ．

♣

リストは**結合**できる．例えばリスト x_{s} とリスト y_{s} を結合したリストは $x_{\mathrm{s}} \mathbin{※} y_{\mathrm{s}}$ と表現する．リストの結合演算子の型は

$$(※) :: [\boldsymbol{a}] \to [\boldsymbol{a}] \to [\boldsymbol{a}] \tag{60}$$

である．[57]

♣

我々は無限リストを持つことができる．例えば自然数を表すリスト n_{s} は $n_{\mathrm{s}} = [1, 2, ...]$ と書くことができる．[58]

無限リストを扱えるのは，我々がいつも遅延評価を行うからである．遅延評価とは，本当の計算は必要になるまで行わないという方式のことである．

もし本当に無限リストを計算機の上で再現する必要があったなら，計算機には無限のメモリが必要になってしまう．しかし我々は，計算が必要になるまで評価を行わないので，無限リストの中から有限個の要素が取り出されるのを待つことができるのである．例えば関数 take m n_{s} はリスト n_{s} から最初の m 個の要素からなるリストを返す．いま

$$x_{\mathrm{s}} = \mathrm{take}\ 5\ n_{\mathrm{s}} \tag{61}$$

とすると，リスト x_{s} は $x_{\mathrm{s}} = [1, 2, ..., 5]$ という値を持つ．[59]

関数 take の型は

$$\mathrm{take} :: \mathbf{Int} \to [\boldsymbol{a}] \to [\boldsymbol{a}] \tag{62}$$

である．

---

[55] Haskell では $[]_{\boldsymbol{a}}$ を `[] a` と書く．これは `[a]` と同じことである．
[56] Haskell では `xs = [x]` と書く．
[57] Haskell では $x_{\mathrm{s}} \mathbin{※} \boldsymbol{y}_{\mathrm{s}}$ を `xs ++ ys` と書く．
[58] Haskell では `ns = [1, 2..]` と書く．
[59] Haskell では `xs = take 5 ns` と書く．

リスト x_{s} の n 番目の要素には $x_{\mathrm{s}} \mathbin{!!} n$ とすることでアクセスできる．[60]

♣

文字列は **Char** 型のリストである．実際 **String** 型は

$$\texttt{type}\ \mathbf{String} = [\mathbf{Char}] \tag{63}$$

と定義されている．[61]

ここにキーワード `type` はデータ型の別名，つまり**型シノニム**を定義するためのキーワードである．

文字列型のリテラルは次のように書く．[62]

$$s = \texttt{"Hello, World!"} \tag{64}$$

リストに対するすべての演算は文字列にも適用可能である．

♣

リストの構成には**内包表記**が使える．例を挙げる．[63]

$$x_{\mathrm{s}} = [x^2 \mid x \in [1, 2...100], \mathrm{even}\ x] \tag{65}$$

関数 even は引数が偶数の場合にだけ True を返す関数である．この例では数列 $[1, 2...100]$ のうち偶数だけを 2 乗したリストを作っている．

マップと畳み込み

リスト x_{s} の各要素に関数 f を適用して，その結果をリスト z_{s} に格納するためには次のように**マップ演算子** $*$ を用いる．[64]

$$z_{\mathrm{s}} = f * x_{\mathrm{s}} \tag{66}$$

式 66 は次の式と同じである．[65]

$$z_{\mathrm{s}} = [fx \mid x \in x_{\mathrm{s}}] \tag{67}$$

リスト x_{s} の各要素を先頭から順番に二項演算子を適用して，その結果を得るには畳み込み演算子を用いる．たとえば整数リストの和は次のように書ける．[66]

$$\begin{aligned} z &= \bigcup_{0}^{\diamond + \diamond} x_{\mathrm{s}} \\ &= a + x_0 + x_1 + ... + x_n \end{aligned} \tag{68}$$

Haskell では $\sum = \cup_0^{\diamond + \diamond}$ として関数 $\sum$ が定義されている．[67]

リスト x_{s} が $x_{\mathrm{s}} = [x_0, x_1, ..., x_n]$ のとき，一般に $\cup_a^{\diamond \circ \diamond}\ x_{\mathrm{s}} = a \circ x_0 \circ x_1 \circ ... \circ x_n$ が成り立つ．ここに $\circ$ は任意の二項演算子である．

---

[60] Haskell では `xs !! n` と書く．
[61] Haskell では `type String = [Char]` と書く．
[62] Haskell では `xs = "Hello, World!"` と書く．
[63] Haskell では `xs = [x ^ 2 | x <- [1, 2..100], x > 50]` と書く．
[64] Haskell では `zs = f <$> x_"s"` と書く．
[65] Haskell では `zs = [f x | x <- x_"s"]` と書く．
[66] Haskell では `z = foldl 0 (+) x_"s"` と書く．
[67] Haskell では $\sum$ 関数を `sum` と書く．

畳み込み演算子には次の右結合バージョンが存在する．[68]

$$
\begin{aligned}
z &= \bigsqcup_{0}^{\lozenge\circ\lozenge} x_{\mathrm{s}} \\
&= a \circ (x_0 \circ (x_1 \circ \ldots \circ (x_{n-2} \circ (x_{n-1} \circ x_n))\ldots))
\end{aligned}
\tag{69}
$$

1.5.2 畳み込み

我々はよくリストの総和を表現するために総和演算子 ($\sum$) を使う．総和演算子とはリスト $[x_0, x_1, \ldots, x_n]$ に対して

$$
\sum [x_0, x_1, \ldots, x_n] = x_0 + x_1 + \ldots + x_n \tag{70}
$$

で定義される演算子である．

この表現を一般化してみよう．リスト $[x_0, x_1, \ldots, x_n]$ が与えられたとき，任意の二項演算子を $\circ$ として

$$
\bigcup_{a}^{\circ} [x_0, x_1, \ldots, x_n] = a \circ x_1 \circ \ldots \circ x_n \tag{71}
$$

であると定義する．

この新しい演算子 $\bigcup$ は**畳み込み演算子**と呼ばれる．変数 a を**アキュムレータ**と呼ぶ．アキュムレータは右側の引数が空であった場合のデフォルト値と考えても良い．[69]

Python 2.7 には畳み込み演算子に相当する `reduce` 関数があり，リスト `ls` の総和 `s` を次のように求めることが出来た．

```
# Python 2.7
ls = [0, 1, 2, 3, 4, 5]
s = reduce(lambda x, y: x + y, ls, 0)
```

この `reduce` 関数は Python バージョン 3 では非推奨になっているが，Ruby には受け継がれていて，Ruby では次のように書ける．

```
# Ruby
ls = [0, 1, 2, 3, 4, 5]
s = ls.inject(0) { |x, y| x + y }
```

♣

リストの総和をとる演算子 $\sum$ は

$$
\sum x_{\mathrm{s}} = \bigcup_{0}^{+} x_{\mathrm{s}} \tag{72}
$$

とすれば得られる．この式は両辺の x_{s} を省略して

$$
\sum = \bigcup_{0}^{+} \tag{73}
$$

とも書く．

[68] Haskell では `foldr` を用いる．
[69] Haskell では $\bigcup_{a}^{+} x_{\mathrm{s}}$ を `foldl (+)) a xs` と書く．

リストの要素のすべての積をとる演算子 $\prod$ は

$$\prod = \bigcup_{1}^{\times} \tag{74}$$

とすれば得られる．

畳み込み演算子は第 1（上）引数に $\boldsymbol{a}$ 型と $\boldsymbol{b}$ 型の引数を取り $\boldsymbol{a}$ 型の戻り値を返す二項演算子，第 2（下）引数に $\boldsymbol{a}$ 型，第 3（右）引数に $\boldsymbol{b}$ 型のリストすなわち $[\boldsymbol{b}]$ 型を取り，$\boldsymbol{a}$ 型の値を返す．従って畳み込み演算子の型は

$$\bigcup :: (\boldsymbol{a} \to \boldsymbol{b} \to \boldsymbol{a} \to \boldsymbol{a} \to [\boldsymbol{b}] \to \boldsymbol{a} \tag{75}$$

である．

畳み込み演算子には次のようなもう一つのバリエーションがある．

$$\bigsqcup_{a}^{\circ}[x_0, x_1, ..., x_n] = (x_0 \circ (x_1 \circ ... \circ (x_n \circ a))) \tag{76}$$

これは**右畳み込み**と呼ばれる演算子である．[70]

畳み込み演算子の面白い応用例を示そう．リストの結合演算子 (※) を使うと

$$\bigcup_{\emptyset}^{※}[[0, 1, 2], [3, 4, 5], ...] = [0, 1, 2, 3, 4, 5, ...] \tag{77}$$

であるから，演算子 $\bigcup_{\emptyset}^{※}$ はリストを平坦化する**平坦化演算子**である．平坦化演算子は concat 演算子とも呼ばれることもあるが，基本的な演算子であるため特別な記号をつけておこう．我々は

$$\flat = \bigcup_{\emptyset}^{※} \tag{78}$$

と定義することにする．[71]

1.5.3 マップ

リストの各要素に決まった関数を適用したい場合がある．Python ではリスト `xs` に関数 `f` を適用するときには次のように書く．

```
# Python
f = lambda x: 100 + x
xs = [1, 2, 3, 4, 5]
ys = map(f, xs)
```

このコードは結果として `ys` に `[101, 102, 103, 104, 105]` を入れる．

このように引数として任意の関数 f と任意のリスト $[x_0, x_1, ..., x_n]$ を取り，戻り値として $[fx_0, fx_1, ..., fx_n]$ を返す演算子 $*$ を考えよう．このとき

$$f * [x_0, x_1, ..., x_n] = [fx_0, fx_1, ..., fx_n] \tag{79}$$

[70] Haskell では `foldr (+) xs a` と書く．引数の順序に注意しよう．
[71] Haskell では演算子 $\flat$ の代わりに `concat` 関数（または `join` 関数）を使う．

であると定義する．この演算子 $*$ をリストの**マップ演算子**と呼ぶ．[72]

リストのマップ演算子の型は

$$(*) :: (\boldsymbol{a} \to \boldsymbol{b}) \to [\boldsymbol{a}] \to [\boldsymbol{b}] \tag{80}$$

である．矢印 $\to$ は右結合なので，これは

$$(*) :: (\boldsymbol{a} \to \boldsymbol{b}) \to ([\boldsymbol{a}] \to [\boldsymbol{b}]) \tag{81}$$

の意味でもある．念のため上式に注釈を加えると

$$(*) :: \underbrace{(\boldsymbol{a} \to \boldsymbol{b})}_{f} \to \left(\underbrace{[\boldsymbol{a}]}_{[x_0, x_1, \ldots, x_n]} \quad \to \quad \underbrace{[\boldsymbol{b}]}_{[fx_0, fx_1, \ldots, fx_n]} \right) \tag{82}$$

である．

ここで f と $(f\,*)$ の型を並べてみると

$$\begin{aligned} f &:: \boldsymbol{a} \to \boldsymbol{b} \\ (f\,*) &:: [\boldsymbol{a}] \to [\boldsymbol{b}] \end{aligned} \tag{83}$$

となり，マップ演算子が何をしているのか一目瞭然になる．

具体例を見てみよう．先程の Python コードの例にあわせて

$$\begin{aligned} f &= \backslash\, x \mapsto 100 + x \\ x_{\mathrm{s}} &= [1, 2, \ldots, 5] \\ y_{\mathrm{s}} &= f * x_{\mathrm{s}} \end{aligned} \tag{84}$$

とすると y_{s} の値は $101, 102, 103, 104, 105]$ となる．

1.5.4 余談：IO サバイバルキット 2

1 行ごとに 3 次元ベクトルが並べられた，以下の入力ファイルがあるとする．

```
1.0 2.0 3.0
4.5 5.5 6.5
```

このようなファイル形式は計算機科学者にとって見慣れたものである．

各行つまり各ベクトルごとに, そのノルムを計算して出力するプログラムを書きたいとしよう．まず数列を受け取ってそのノルムを返す関数 norm を次のように定義する．

$$\begin{aligned} &\mathrm{norm} :: [\mathbf{Double}] \to \mathbf{Double} \\ &\mathrm{norm}\ \emptyset = 0.0 \\ &\mathrm{norm}\ x_{\mathrm{s}} = \mathrm{sqrt}\ \left(\sum [x * x \mid x \in x_{\mathrm{s}}] \right) \end{aligned} \tag{85}$$

式 85 の 1 行目に出てくる型 [**Double**] は **Double** のリストである．

式 85 は関数のパタンマッチングを使っている．引数が空リストつまり $\emptyset$ の場合，関数は 0.0 を返す．それ以外の場合は，リスト内包表記を使ってノルムを計算して返す．

入力ファイル全体を受け取るにはアクション **getContents** を用いる．

[72] Haskell では `map f xs` または `f <$> xs` と書く．演算子 `<$>` は `fmap` 演算子の中置バージョンである．

入力ファイルを 1 行毎のリストにするには関数 lines を用いる．関数 lines の型は

$$\text{lines} :: \textbf{String} \to [\textbf{String}] \tag{86}$$

である．関数 lines を用いると，入力 `"1.0 2.0 3.0\n 4.5 5.5 6.5"`が `["1.0 2.0 3.0", "4.5 5.5 6.5"]` になる．

各行を空白で区切ってリストに格納するには関数 words を用いる．関数 words の型は

$$\text{words} :: \textbf{String} \to [\textbf{String}] \tag{87}$$

である．関数 words を用いると，入力 `"1.0 2.0 3.0"` が `["1.0", "2.0", "3.0"]` になる．

文字列を浮動小数点数に変換するには次の関数 readDouble を用いる．

$$\begin{aligned} \text{readDouble} &:: \textbf{String} \to \textbf{Double} \\ \text{readDouble} &= \text{read} \end{aligned} \tag{88}$$

この関数 readDouble は標準関数 read に型注釈を付けただけのものであるが，今回の目的には十分である．例えば `readDouble "1.0"` は `1.0` を返す．

入力ファイルの各行に書かれたベクトルを対象に関数 `norm` を適用して，結果を書き出すには，これまでの関数を結合して次のように書く．

$$\textbf{main} = \textbf{print} \circ (\texttt{norm}\,*) \circ ((\text{readDouble}\,*)\,*) \circ (\text{words}\,*) \circ \text{lines} \heartsuit \textbf{getContents} \tag{89}$$

式 89 は，右辺の右端から次のように読むことが出来る．

1. 入力全体を **getContents** で読み込む．
2. 読み込んだ入力を lines で 1 行毎のリストに変換する．
3. 1 行毎のリストを words で空白で区切ったリストに変換する．
4. 空白で区切ったリストを readDouble で浮動小数点数に変換する．
5. 浮動小数点数のリストのノルムを `norm` で計算する．
6. ノルムのリストを **print** で出力する．

式 89 を Haskell コードで書くと次のようになる．

```haskell
-- Haskell
norm :: [Double] -> Double
norm [] = 0.0
norm xs = sqrt (sum [x * x | x <- xs])

readDouble :: String -> Double
readDouble = read

main :: IO ()
main = print
          . (norm <$>)
          . ((readDouble <$>) <$>)
          . (words <$>)
          . lines
          =<< getContents
```

式 89 において，アクション **print** に代えて次の **printEach** を用いると，入力と出力を同じ形式にできる．[73]

[73] Haskell では `printEach xs = print `mapM` xs` と書く．

$$\textbf{printEach}\ x_{\mathrm{s}} = \textbf{print} *_{\mathrm{M}}\ x_{\mathrm{s}} \tag{90}$$

演算子 $*_{\mathrm{M}}$ はアクション版のマップ演算子である．

1.5.5 この章のまとめ

📍 **TK** この章のまとめ

1.6 再帰

📍 **TK** 再帰

1.6.1 関数の再帰適用

自然数 n の**階乗**は次のように定義される．

$$n! = n \times (n-1) \times \ldots \times 1 \tag{91}$$

数学者は $0! = 1$ と定義するので，この式は

$$\begin{aligned} 0! &= 1 \\ n! &= n \times (n-1)! \end{aligned} \tag{92}$$

という風に「**再帰**的に」書き直すことが出来る．再帰とは，定義式の両辺に同じ関数名が現れることを指す．

Haskell で書きやすいように，後置演算子 ! のかわりに関数 fact を定義しよう．関数は内部で自分自身を適用しても良いので，階乗関数 fact の定義は次のようになる．

$$\mathrm{fact}\ x = \texttt{case}\ x\ \texttt{of}\ \begin{cases} 0 \dashrightarrow 1 \\ \square \dashrightarrow x \times \mathrm{fact}(x-1) \end{cases} \tag{93}$$

と定義できる．関数が自分自身を適用することを関数の**再帰適用**と呼ぶ．[74]

これで我々は関数の適用，変数の代入，ラムダ式，条件式，再帰の方法を学んだわけである．これだけあれば，原理的にはどのようなアルゴリズムも書くことができる．今日からはカリー風な数学であらゆるアルゴリズムを表現できるのである！

♣

このように関数は再帰的に呼び出せる．いま $n \geq 0$ を前提とすると n 番目のフィボナッチ数を計算する関数 fib を次のように定義できる．

$$\begin{aligned} \mathrm{fib}\ 0 &= 0 \\ \mathrm{fib}\ 1 &= 1 \\ \mathrm{fib}\ n &= \mathrm{fib}\ (n-1) + \mathrm{fib}\ (n-2) \end{aligned} \tag{94}$$

1.6.2 リストと再帰

数学におけるリストは自由に考えることが出来るが，計算機上ではその実装も考えておかねばならない．我々はリストを Lisp におけるリスト構造と同じ構造を持つものとする．Lisp におけるリストとは，変数 first と 変数 rest からなるペアの集合である．変数 first がリストの要素を参照し，変数 rest が次のペアを参照する．リストの最後のペアの rest は空リストを参照する．

[74] Haskell では `fact x = case x of { 0 -> 1; _ -> x * fact (x - 1) }` と書く．

リストのための特別な表現

$$\text{first} : \text{rest} \tag{95}$$

を用い，変数 first はリストが保持する型，変数 rest はリスト型であるとする．演算子 : を Lisp に倣って **cons 演算子**と呼ぶ．

要素 rest はリストまたは空リストであるから，一般にリストは次のように展開できることになる．

$$\begin{aligned}[x_0, x_1, x_2, ..., x_n] &= x_0 : [x_1, x_2, ..., x_n] \\ &= x_0 : x_1 : [x_2, ..., x_n] \\ &= x_0 : x_1 : x_2 : ... : x_n : \emptyset\end{aligned} \tag{96}$$

cons 演算子 (:) は右結合する．すなわち $x_0 : x_1 : x_2 = x_0 : (x_1 : x_2)$ である．

マップ演算子 $(*)$ の実装は，リストの実装に踏み込めば簡単である．空でないリストは必ず $x : x_\mathrm{s}$ という形をしているから，次のようにマップ演算子を定義できる．

$$\begin{aligned}f * \emptyset &= \emptyset \\ f * (x : x_\mathrm{s}) &= (fx) : (f * x_\mathrm{s})\end{aligned} \tag{97}$$

♣

Haskell では任意のリスト x_s に対し，次の関数が用意されている．

$$\begin{aligned}\text{head } x_\mathrm{s} &... \textbf{先頭要素} \\ \text{tail } x_\mathrm{s} &... \textbf{2 番目以降の要素からなるリスト}\end{aligned} \tag{98}$$

これらは Lisp の `car` 関数，`cdr` 関数と同じものであり，この二者を用いればどのようなリストの処理も可能である．[75]

このように基本的な関数から高機能な関数を実装する方法はよく行われる．この例では cons 演算子からマップ演算子を合成した．

♣

関数はリストを受け取ることができる．次の書き方では，関数 f は整数リストの最初の要素 x と残りの要素 x_s を別々に受け取り，先頭要素だけを返す．[76]

$$\begin{aligned}f &:: [\mathbf{Int}] \to \mathbf{Int} \\ f(x : x_\mathrm{s}) &= x\end{aligned} \tag{99}$$

ただし，引数のリストが空リストである可能性を考慮して，式 99 は次のように書き直すべきである．

$$\begin{aligned}f &:: [\mathbf{Int}] \to \mathbf{Int} \\ f\emptyset &= 0 \\ f(x : x_\mathrm{s}) &= x\end{aligned} \tag{100}$$

$f\emptyset$ が 0 を返すのは不自然だが，関数fの戻り型を整数型としているためこれは仕方がない．エラーを考慮する場合は節 1.7 で述べる Maybe を使う必要がある．

[75] Haskell では `head` 関数の利用は非推奨である．これは `head []` と書くとエラーになるからである．

[76] この関数 f の実装は Haskell の `head` 関数と同じである．Haskell では `head` 関数の使用は非推奨となっており，代わりに `headMaybe` 関数が推奨されている．

リストの再帰を使うと，クイックソートも簡単に書くことが出来る．クイックソートは次のように定義できる．

$$\begin{aligned} \text{quicksort } \emptyset &= \emptyset \\ \text{quicksort } (x : x_{\mathrm{s}}) &= \text{quicksort } l_{\mathrm{s}} \mathbin{※} [x] \mathbin{※} \text{quicksort } r_{\mathrm{s}} \\ &\qquad \texttt{where } \{l_{\mathrm{s}} = [l \mid l \in x_{\mathrm{s}}, l < x]; r_{\mathrm{s}} = [r \mid r \in x_{\mathrm{s}}, r \geq x]\} \end{aligned} \tag{101}$$

リストを引数にとる関数はいつでも $f(x : x_{\mathrm{s}}) = \ldots$ という風にパタンマッチを行えるが，式の右辺でリスト全体すなわち $(x : x_{\mathrm{s}})$ を参照したい場合もあるであろう．そのような場合は

$$f a_{\mathrm{s}} \mathbin{@} (x : x_{\mathrm{s}}) = \ldots \tag{102}$$

として，変数 a_{s} でリスト全体を参照することも可能である．このような記法を **as パタン**と呼ぶ．[77]

なお，関数は再帰させるたびに計算機のスタックメモリを消費する．これを回避するためのテクニックが，次節で述べる末尾再帰である．

1.6.3 末尾再帰

計算機科学者は，同じ再帰でも**末尾再帰**という再帰のスタイルを好む．末尾再帰とは，関数の再帰適用を関数定義の末尾にすることである．この章に出てきた階乗関数 fact を例にとろう．階乗関数 fact は

$$\text{fact } x = \texttt{case } x \texttt{ of } \begin{cases} 0 \dashrightarrow 1 \\ \square \dashrightarrow x \times \text{fact}(x-1) \end{cases} \tag{103}$$

のような形をしていた．末尾の関数をよりはっきりさせるために演算子 $(*)$ を前置にして

$$\text{fact } x = \texttt{case } x \texttt{ of } \begin{cases} 0 \dashrightarrow 1 \\ \square \dashrightarrow (\times)x \;\; \text{fact}(x-1) \end{cases} \tag{104}$$

と書いてみよう．この定義の末尾の式は $(\times)x \;\; \text{fact}(x-1)$ である．これだと末尾の関数は fact ではなく演算子 $(\times)$ なので，末尾に再帰適用を行ったことにはならない．

そこで，次のように形を変えた階乗関数 fact′ を考えてみる．

$$\text{fact}' a x = \texttt{case } x \texttt{ of } \begin{cases} 0 \dashrightarrow a \\ \square \dashrightarrow \text{fact}'(a \times x)\ (x-1) \end{cases} \tag{105}$$

こうすれば末尾の関数がもとの fact′ と一致する．[78]

関数 fact が，例えば

$$\begin{aligned} \text{fact } 3 &= 3 \times \text{fact } 2 \\ &= 3 \times 2 \times \text{fact } 1 \\ &= 3 \times 2 \times 1 \times \text{fact } 0 \\ &= 3 \times 2 \times 1 \times 1 \\ &= 6 \end{aligned} \tag{106}$$

と展開されるのに対し，同じく fact′ 3 は

[77] Haskell では `f as @ (x : xs) = ...` と書く．

[78] Haskell では `fact' a x = case x of { 0 -> 1; _ -> fact' (a*x) (x-1) }` と書く．

$$
\begin{aligned}
\mathrm{fact}'\ 1\ 3 &= \mathrm{fact}'(1 \times 3)\ (3-1) \\
&= \mathrm{fact}'\ 3\ 2 \\
&= \mathrm{fact}'(3 \times 2)\ (2-1) \\
&= \mathrm{fact}'\ 6\ 1 \\
&= \mathrm{fact}'(6 \times 1)\ (1-1) \\
&= \mathrm{fact}'\ 6\ 0 \\
&= 6
\end{aligned}
\tag{107}
$$

であるから，関数 fact が「横に伸びる」のに対して，関数 fact′ は「横に伸びない」ことになる．計算式が「横に伸びない」性質は，計算機のリソースを無駄に消耗しないことが期待されるため，計算機科学者が好むのである．また末尾再帰は後述する「末尾再帰最適化」のチャンスをコンパイラに与える．

Haskell を含む幾つかのプログラミング言語処理系は，コンパイル時に**末尾再帰最適化**を行う．末尾再帰最適化とは，一言で言うと再帰を計算機が扱いやすいループに置き換えることである．では最初から我々もループで関数を表現しておけば，と思われるかもしれないが，再帰以外の方法でループを表現する場合には必ず変数（ループカウンタ）への破壊的代入が必要になるため，我々は末尾再帰に慎ましくループを隠すのである．[79]

1.6.4 余談：遅延評価

Haskell は，意図しない限り遅延評価を行う．これは特に左畳み込み演算子 $\cup$ を使う場合に問題となる．いま $x_{\mathrm{s}} = [x_0, x_1, x_2, x_3]$ とすると，左畳み込み演算 $\cup_0^+\ x_{\mathrm{s}}$ は

$$
\begin{aligned}
\bigcup_{0}^{+} x_{\mathrm{s}} &= \cup_0^+\ (x_0 : x_1 : x_2 : \emptyset) \\
&= \bigcup_{0+x_0}^{+} (x_1 : x_2 : x_3 : \emptyset) \\
&= \bigcup_{(0+x_0)+x_1}^{+} (x_2 : x_3 : \emptyset) \\
&= \bigcup_{((0+x_0)+x_1)+x_2}^{+} (x_3 : \emptyset) \\
&= \bigcup_{(((0+x_0)+x_1)+x_2)+x_3} \emptyset \\
&= (((0 + x_0) + x_1) + x_2) + x_3
\end{aligned}
\tag{108}
$$

と展開される．遅延評価のために，Haskell 処理系は値ではなく式をメモリにストアしなければならないが，左畳み込み演算は大きなメモリを必要としがちである．もし例えば予め $0 + x_0$ を先に計算しておくなど左畳み込みだけ先に評価しておけば，大いにメモリの節約になる．そのために Haskell は「遅延評価無し」の左畳み込み演算子を用意している．[80]

1.6.5 この章のまとめ

📍 **TK** この章のまとめ

[79] Scheme は末尾再帰最適化を行うことが言語仕様によって決められている．

[80] 「遅延評価無し」の左畳み込み演算子を Haskell では `foldl'` と書く．

1.7 Maybe

この章では計算結果が正しいかもしれないし，正しくないかもしれないという曖昧な状況を表す型を導入する．手始めに Python でクラス `Possibly` を実装し，それがカリー風の数式で綺麗に書けることを示す．またリストとの共通点についても見ていくことにする．

1.7.1 Possibly

計算は失敗する可能性がある．たとえば

$$z = \frac{y}{x} \tag{109}$$

のときに $x = 0$ であったとしたら，この計算は失敗する．プログラムが計算を失敗した場合，たいていのプログラマは大域ジャンプを試みる．しかし大域ジャンプは変数の書き換えを行うことであるから，別の方法が望まれる．Haskell では失敗する可能性がある場合には Maybe という機構が使える．

計算の途中で，計算にまつわる状態を残りの計算に引き継ぎたくなる場合がある．例えば，整数 x, y, z があり $z = {}^{y}/_{x}$ なる値を続く計算で利用したいとする．だが $x \equiv 0$ のときには z は正しく計算されない．こんなときプログラマが取れる手段はだいたい次のとおりだろう．

- $z = {}^{y}/_{x}$ を計算した時点で**ゼロ除算例外**を発生させ，プログラムの制御を他の場所へ移す（大域ジャンプを行う）
- グローバル変数にゼロ除算エラーが起こったことを記録しておき，z にはとりあえずの数値，例えば 0 を代入しておいて，計算を続行させる
- z にエラー状態を示す印を新たにつけておいて，計算を続行させる

大域ジャンプも，グローバル変数の書き換えも破壊的代入を伴うものであり，受け入れがたい．そこで我々は第三のエラー状態を示す印をつける方法を採用することにする．通常，変数が整数だろうが実数だろうが，計算機表現には余分なビットが残っていないので，変数をラップする次のようなクラス `Possibly` を導入することにしよう．メンバ変数 `value` が値を，メンバ変数 `valid` がエラーの有無を表す．

```
# Python
class Possibly:
  def __init__(self, a_valid, a_value = 0):
    self.valid = a_valid
    self.value = a_value
```

例えば整数値 `123` を持つ `Possibly` クラスの値 `p` は次のように生成できる．

```
# Python
p = Possibly(True, 123)
```

`Possibly` 値 `p` が計算エラーを表す場合は次のように初期化出来る．

```
# Python
p = Possibly(False)
```

ここで，引数に `1` を加えて返す関数 `f` があるとしよう．関数 `f` の定義は次の通りである．

```
# Python
f = lambda x: 1 + x
```

関数 f に直接 Possibly 値 p を食わせるとランタイムエラーを引き起こす．

```
# Python
q = f(p) # エラー!!
```

これは関数 f が引数として数値を期待していたにもかかわらず Possibly クラスの値が渡されたからである．もし関数 f のほうをいじりたくないとすれば，次のような関数 map_over を使って間接的に関数適用を行う必要がある．

```
# Python
q = map_over(f, p)
```

関数 map_over(f, p) はもし p がエラーを表す値でなければ中身の値を関数 f に適用し，その結果を Possibly クラスに包んで返す．もし p がエラー値を表す値であれば，結果もエラー値である．関数 map_over の実装は次のようになる．

```
# Python
def map_over(f, p):
  if p.valid == True:
    return Possibly(True, f(p.value))
  else:
    return Possibly(False)
```

さて，次節では以上のようなことを抽象数学的に綺麗に描いてみよう．

1.7.2 Maybe

もう一度振り出しに戻る．

いま関数 f が引数 x と y を取り，$x \neq 0$ であるならば y/x を返すとする．もし $x \neq 0$ であれば失敗を意味する Nothing（ナッシング）を返すとする．すると関数 f の定義は次のようになる．

$$fxy = \texttt{if}\ x \neq 0\ \texttt{then}\ y/x\ \texttt{else}\ \text{Nothing}\ \ldots\ \textbf{不完全な定義} \tag{110}$$

残念ながら上式は不完全である．なぜならば $x \neq 0$ のときの戻り値は数であるのに対して，$x \neq 0$ のときの戻り値は数ではないからである．そこで

$$fxy = \texttt{if}\ x \neq 0\ \texttt{then}\ \text{Just}_{(y/x)}\ \texttt{else}\ \text{Nothing} \tag{111}$$

とする．ここに $\text{Just}_{(y/x)}$ は数 y/x から作られる，Maybe で包まれた数である．[81]

この関数 f の戻り値はもはや整数 (**Int**) 型とは言えない．そこでこの型を $\mathbf{Maybe_{Int}}$ と表して「Maybe 整数（おそらく整数）」型と呼ぶことにする．型 $\boldsymbol{a}$ から型 $\mathbf{Maybe_{\boldsymbol{a}}}$ を生成するとき，**Maybe** を**型コンストラクタ**と呼ぶ．[82]

Maybe で包まれた型を持つ変数は $z_?$ のように小さく ? をつける．例を挙げる．[83]

$$z_? :: \mathbf{Maybe_{Int}} = \text{Just}_1 \tag{112}$$

Maybe 変数が値を持たない場合は $x_? = \text{Nothing}$ のように書く．[84]

[81] Haskell では `f x y = if x /= 0 then Just y / x else Nothing` と書く．
[82] Haskell では `Maybe a` と書く．
[83] Haskell では `xm :: Maybe Int = Just 1` と書く．
[84] Haskell では `xm = Nothing` と書く．

ここで変数 $z_?$ が取り得る値は正しく計算された値をラップしたものか，あるいはエラーを表す値 Nothing である．このように計算結果に「意味付け」をすることを**文脈**に入れると言う．

一度 Maybe になった変数を非 Maybe に戻すことは出来ない．変数 z が一度ゼロ除算の危険性に「汚染」された場合，その後ずっと Maybe 変数に入れ続けなければいけない．そこで，普通の変数を引数にとる関数 f に Maybe 変数 $z_?$ を食わせるには，リストの時と同じようなマップ演算子が必要になる．具体的には，変数 x が **Int** 型として，Maybe 変数 $w_? = \mathrm{Just}_x$ が与えられたとき

$$f \circledast_? w_? = \mathrm{Just}_{fx} \tag{113}$$

となるような Maybe バージョンのマップ演算子 $\circledast_?$ を用いる．ここに $f \circledast_? w_?$ の型は，もし $f :: \mathbf{Int} \to \mathbf{Float}$ ならば $\mathbf{Maybe}_{\mathbf{Float}}$ である．[85]

実際には $w_? \equiv \mathrm{Nothing}$ の可能性も考えなければならないから，Maybe バージョンのマップ演算子は

$$f \circledast_? w_? = \texttt{case}\ w_?\ \texttt{of} \begin{cases} \mathrm{Just}_x \dashrightarrow \mathrm{Just}_{fx} \\ \square \dashrightarrow \mathrm{Nothing} \end{cases} \tag{114}$$

でなければならない．この Maybe バージョンのマップ演算子 $\circledast_?$ は

$$\begin{aligned} f \circledast_? \mathrm{Just}_x &= \mathrm{Just}_{fx} \\ f \circledast_? \mathrm{Nothing} &= \mathrm{Nothing} \end{aligned} \tag{115}$$

と定義すれば得られる．[86]

今後，普通の（引数に Maybe が来ることを想定していない）関数 f を Maybe 型である変数 $w_?$ に適用させるときには，必ず

$$w_? = f \circledast_? w_? \tag{116}$$

のように Maybe バージョンのマップ演算子 $\circledast_?$ を用いることにする．これはプログラムの安全性のためである．変数が一旦ゼロ除算の可能性に汚染されたら，最後まで Maybe に包んでおかねばならない．

Python で Maybe の概念を忠実になぞることは難しい．と言うのも Python は動的型付け言語であるため，型コンストラクタという概念が無いからだ．一方で Maybe の概念を静的型付け言語である C++や Java で実現することはできる．そこで C++の本物のコードで示しておこう．ただしポインタを使わないでおいたので C++プログラマも Java プログラマも参考にできるだろう．[87]

Maybe は次の `maybe` クラステンプレートで表現できる．（Java プログラマへの注意：これは `maybe<a>` クラスの定義と同じ意味である．）

```
// C++
template <typename a> class maybe {
  private:
    a value;
    bool valid;
  public:
```

[85] Haskell では $f \circledast_? w_?$ を `f <$> w` と書く．

[86] Haskell では `f <$> Just x  = Just (f x); f <$> Nothing = Nothing` と書く．

[87] C++17 にはまさにこの目的のための標準テンプレート `template <typename T> std::optional<T>` がある．

```
    maybe(): value(0), valid(false) { }  // デフォルトコンストラクタ
    maybe(a a_value): value(a_value), valid(true) { }
    a get_value() const { return value; }
    bool is_valid() const { return valid; }
};
```

デフォルトコンストラクタ `maybe()` は例外的な状況を表す Nothing を生成し，1 引数コンストラクタ `maybe(a)` は `maybe<a>` で包んだ引数値を生成する．[88]

当然我々には Maybe バージョンのマップ演算子が必要である．ここでは関数 `map_over` として書いてみよう．(Java プログラマへの注意 :関数 `map_over` はどのクラスにも属していないが，それで正解なのである．)

```
// C++
template <typename a, typename b, typename fn>
maybe<b> map_over(fn f, maybe<a> w) {
  if (w.is_valid()) {
    return maybe<b>(f(w.get_value()));
  }
  else {
    return maybe<b>();
  }
}
```

テンプレートの 3 番目の引数 `fn` は関数 `f` を受け取るために必要である．C++はコンパイル時までにすべての変数の型が決定していないといけないが，関数 `f` の型は関数 `map_over` 設計時には確定できないため，このようにテンプレートにしている．

整数 x から Maybe 値 $u_? = \mathrm{Just}_x$ を作り，関数 $gx = 1 + x$ を Maybe 値 $u_?$ に食わせて Maybe 値 $v_?$ ただし $v_? = g \circledast_? u_?$ を得ることを C++では次のように書くことになる．

```
// C++
int x = 123;
maybe<int> u(x);
auto g = [](int x) -> int { return 1 + x; };
maybe<int> v = map_over(g, u);
```

注意してほしいのは `g(x)` も `map_over(g, u)` も正当なコードだが `g(u)` は型エラーであることだ．また `g(u.get_value())` は正当なコードだが，わざわざ `u` が持つ文脈を捨てることになる．

♣

リストが Maybe の中に入っている場合は，リストの各要素に関数を適用することができる．例を挙げる．

$x_? = \mathrm{Just}_{[1,2\ldots100]}$ のとき，リストの各要素に関数 $f :: \mathbf{Int} \to \mathbf{Int}$ を適用するには次のように書く．[89]

$$z_? = (f\,*) \circledast_? x_? \tag{117}$$

[88] この C++コードは，型 `a` が `0` を持てることを前提に書かれているため，Haskell の Maybe とは厳密には異なる．

[89] Haskell では `zm = (f <$>) <$> xm` と書く．最初の `<$>` はリストの各要素に関数 `f` を適用する演算子，2 番目の `<$>` は Maybe の中のリストの各要素に関数 `f` を適用する演算子である．

1.7.3 リストと Maybe

関数 f を Mayby 値 $u_?$ に適用するために

$$v_? = f \circledast_? u_? \tag{118}$$

のような Maybe バージョンのマップ演算子 $(\circledast_?)$ を使った．一方で，同じ関数 f をリスト x_s に適用するには

$$y_\mathrm{s} = f * x_\mathrm{s} \tag{119}$$

のようなリストバージョンのマップ演算子 $(*)$ を使った．

リストバージョンのマップ演算子 $(*)$ をもし C++で書くとしたら，次のようなコードになる．ここでリスト型として C++の標準テンプレートライブラリ（STL）の `std::list` クラスを流用した．

```
// C++
template <class a, class b, class fn>
std::list<b> map_over(fn f, std::list<a> xs) {
  std::list<b> y_s(xs.size());
  auto i = xs.cbegin();
  auto j = ys.begin();
  while (i != xs.cend()) {
    *j = f(*i); ++i; ++j;
  }
  return std::list<b>(ys);
}
```

この関数 `map_over` の中身部分はどうでもよろしい．それよりも，リストバージョンのマップ演算子の C++関数のインタフェースと，Maybe バージョンのマップ演算子の C++関数のインタフェースを見比べてみよう．

```
// C++
// リスト
template <class a, class b, class fn>
std::list<b> map_over(fn f, std::list<a> xs);
// Maybe
template <class a, class b, class fn>
maybe<b> map_over(fn f, maybe<a> w);
```

やはりそっくりである．であるならば，うまく統一したい．C++では次のような書き方が文法的には可能である．

```
// C++
template <class a, class b, template<class> X, class fn>
X<b> map_over(fn f, X<a> x);
```

これは一見上手く行きそうに見えるが，このコードは `map_over` のインスタンス化で躓くため，次のように `b` 型のダミー変数が必要になる．

```
// C++
template <class a, class b, template<class> X, class fn>
X<b> map_over(fn f, X<a> x, b dummy);
```

残念なことに，いずれのコードにしてもリストと Maybe の本質的な抽象化にはなってない．型 `x` がマップ可能なコンテナであることをテンプレート機構を使って保証することができないためである．この問題は C++26 で導入予定の「コンセプト」機能によって解決する見込みである．

一方で，数学者たちが見つけた圏という代数的構造が，リストも Maybe も統一的に扱うことを可能にしている．これを発見したのは Eugenio Moggi を始めとする計算機科学者たちである．この人類の英知は節 1.8 から見ていくことにしよう．

1.7.4 余談: Either

Maybe とよく似た型に Either がある．Maybe が $\boldsymbol{a}$ 型または Nothing のいずれかの値をとったように，Either は $\boldsymbol{a}$ 型または $\boldsymbol{b}$ 型のいずれかの値を取る．$\boldsymbol{a}$ 型または $\boldsymbol{b}$ 型を取る Either 型の変数 $e_!$ があるとすると，

$$e_! :: \mathbf{Either}_{\boldsymbol{ab}} \tag{120}$$

と書く．Either 型の生成は型 $\boldsymbol{a}$ および $\boldsymbol{b}$ から型コンストラクタを用いて $\mathbf{Either}_{\boldsymbol{ab}}$ と書く．[90]

Either には値コンストラクタが 2 種類あり，それぞれ Right_x と Left_x である．値コンストラクタは

$$e_! = \mathrm{Right}_x \tag{121}$$

または

$$e_! = \mathrm{Left}_x \tag{122}$$

のように使う．[91]

Either はより複雑な計算エラーが発生する場合に用いる．Maybe が単に失敗を表す Nothing しか表現できなかったのに対し，Either は任意の型の変数で表現できる．習慣的に，正しい (right) 計算結果は Right_x 値コンストラクタで格納し，残された (left) エラーの情報は Left_x 値コンストラクタで格納する．

Either 型は C の共有型 (`union`) や C++のバリアント型 (`std::variant`) に近い．

1.7.5 この章のまとめ

📍 **TK** この章のまとめ

1.8 関手

📍 **TK** 関手

1.8.1 リストの世界・Maybe の世界

$\boldsymbol{a}$ 型の変数 $x, y :: \boldsymbol{a}$ について，関数 $f :: \boldsymbol{a} \to \boldsymbol{a}$ があり

$$y = fx \tag{123}$$

であるとしよう．このように型 $\boldsymbol{a}$ で閉じた世界を仮に $\boldsymbol{a}$ 世界と呼ぶことにする．この世界では，関数 f は値 x を値 y に変換する．

[90] Haskell では `Either a b` と書く．
[91] Haskell ではそれぞれ `e = Right x` および `e = Left x` と書く．

同様に $\mathbf{Maybe}_{\boldsymbol{a}}$ 型の変数 $u_?, v_? :: \mathbf{Maybe}_{\boldsymbol{a}}$ について，関数 $\varphi :: \mathbf{Maybe}_{\boldsymbol{a}} \to \mathbf{Maybe}_{\boldsymbol{a}}$ があり

$$v_? = \varphi u_? \tag{124}$$

であるとしよう．このように型 $\mathbf{Maybe}_{\boldsymbol{a}}$ で閉じた世界を仮に $\mathbf{Maybe}_{\boldsymbol{a}}$ 世界と呼ぶことにする．この世界では，関数 φ は値 $u_?$ を値 $v_?$ に変換する．

ここで，変数 x, y と Maybe 変数 $u_?, v_?$ は値コンストラクタによって

$$\begin{aligned} u_? &= \mathrm{Just}_x \\ v_? &= \mathrm{Just}_y \end{aligned} \tag{125}$$

の関係にあるとしよう．値コンストラクタは値を $\boldsymbol{a}$ 世界から $\mathbf{Maybe}_{\boldsymbol{a}}$ 世界へとジャンプさせる機能を持っている．

他に $\boldsymbol{a}$ 世界から $\mathbf{Maybe}_{\boldsymbol{a}}$ 世界へとジャンプさせるものがあるだろうか．よく考えてみると，マップ演算子もそうである．いま $u_? = \mathrm{Just}_x, v_? = \mathrm{Just}_y$ なのだから，$\boldsymbol{a}$ 世界の関数 f と $\mathbf{Maybe}_{\boldsymbol{a}}$ 世界の関数 φ は無関係ではなく

$$v_? = \varphi u_? = f \circledast_? u_? \tag{126}$$

であり，

$$\varphi = f \circledast_? \tag{127}$$

である．つまりマップ演算子 $\circledast_?$ が関数 f を $\boldsymbol{a}$ 世界から $\mathbf{Maybe}_{\boldsymbol{a}}$ 世界へとジャンプさせているのである．ジャンプを次の表で波矢印 ($\rightsquigarrow$) で表した．

世界（圏）	値（対象）	写像（射）	値のジャンプ	関数のジャンプ
$\boldsymbol{a}$ 世界	x, y	$y = fx$		
$\mathbf{Maybe}_{\boldsymbol{a}}$ 世界	$u_?, v_?$	$v_? = \varphi u_?$	$x \rightsquigarrow u_?, y \rightsquigarrow v_?$	$f \rightsquigarrow \varphi$

いま「世界」と呼んだものを，数学者は**圏**と呼ぶ．圏とは**対象**と**射**の組み合わせである．本書では「対象」とは変数のことであり，射とは関数だと思えば良い．そして，圏から圏へとジャンプさせるもの ($\rightsquigarrow$) を**関手**と呼ぶ．この例で言えば値コンストラク Just_x とマップ演算子 $\circledast_?$ が関手である．値コンストラクタ Just_x は $\boldsymbol{a} \to \mathbf{Maybe}_{\boldsymbol{a}}$ という型を持ち，マップ演算子 $\circledast_?$ は $(\boldsymbol{a} \to \boldsymbol{b}) \to (\mathbf{Maybe}_{\boldsymbol{a}} \to \mathbf{Maybe}_{\boldsymbol{b}})$ という型を持つ．[92]

♣

Haskell ではマップ演算子が定義された型を関手型と呼ぶ．具体的には，マップ演算子が定義されたすべての型は Functor 型クラスのインスタンスであるとする．つまり Functor 型クラスには一般化されたマップ演算子が定義されており，そのインスタンスであるリストや Maybe は独自のマップ演算子を定義しなければならないということである．

一般化されたマップ演算子を $\circledast$ で表そう．この $\circledast$ 演算子は

$$(\circledast) :: \mathsf{Functor} \supset \boldsymbol{f} \Rightarrow \boldsymbol{f}_{\boldsymbol{a}} \to \boldsymbol{f}_{\boldsymbol{b}} \tag{128}$$

という型を持つ．ここに $\mathsf{Functor} \supset \boldsymbol{f}$ は，$\boldsymbol{f}$ が Functor 型クラスに属すという制約を表してい

[92] 関手は英語でファンクター(functor)と言うが，C++の関数オブジェクト（function object）もかつてはファンクター（functor）と呼ばれていた．C++のファンクターとはクロージャの代用品のことで，本書で述べる関手とは異なる概念である．混同しないように注意しよう．

る．また $\boldsymbol{f}$ は型コンストラクタであり，$\boldsymbol{f_a}$ は $\boldsymbol{f}$ 型コンストラクタと $\boldsymbol{a}$ 型によって作られたコンテナ型である．

もし型コンストラクタがリスト型コンストラクタであれば，つまり $\boldsymbol{f} = []$ であれば

$$(\circledast) :: (\boldsymbol{a} \to \boldsymbol{b}) \to [\boldsymbol{a}] \to [\boldsymbol{b}] = *\tag{129}$$

であるし，もし型コンストラクタが Maybe 型コンストラクタであれば，つまり $\boldsymbol{f} = \mathbf{Maybe}$ であれば

$$(\circledast) :: (\boldsymbol{a} \to \boldsymbol{b}) \to \mathbf{Maybe}_{\boldsymbol{a}} \to \mathbf{Maybe}_{\boldsymbol{b}} = \circledast_?\tag{130}$$

である．

型	型コンストラクタ	マップ演算子	変数の具体例
$[\boldsymbol{a}]$	$[]$	$*$	$[x]$
$\mathbf{Maybe}_{\boldsymbol{a}}$	$\mathbf{Maybe}$	$\circledast_?$	Just_x
Functor $\supset \boldsymbol{f} \Rightarrow \boldsymbol{f_a}$	$\boldsymbol{f}$	$\circledast$	具体的な型による

リストと Maybe は両者ともマップ演算子（と値コンストラクタ）を持つ．両者の関係をまとてみたのが表〜〜〜である．オブジェクト指向プログラマなら，リストと Maybe に共通のスーパークラスを設計したくなるであろう．それが型クラス Functor である．

♣

別の言い方をすると，我々はオブジェクト指向プログラミングよりもエレガントな方法で，リストと Maybe の共通項をくくりだすことにする．いよいよ型クラスの出番である．

リスト型 $[\boldsymbol{a}]$ も Maybe 型 $\mathbf{Maybe}_{\boldsymbol{a}}$ も Functor 型クラスに属すのであった．そこで Functor 型クラスは**一般マップ演算子** $(\circledast)$ を持つものとする．

一般マップ演算子 $(\circledast)$ は**多様的**である．この意味は，もし $f \circledast x_{\mathrm{s}}$ と書いてあれば $f * x_{\mathrm{s}}$ のことであるし，もし $f \circledast x_?$ と書いてあれば $f \circledast_? x_?$ のことであると自動的に解釈することである．そして，何の飾りもつけられていない変数 x がふらっと現れ，目の前に $f \circledast x$ という式が登場しても，落ち着いて変数 x の型を調べ，変数 x がリストならば $\circledast$ の部分に $*$ を，変数 x が Maybe ならば $\circledast$ の部分に $\circledast_?$ をはめ込むのだ．[93]

♣

型 $\boldsymbol{a}$ のリストの変数は $x_{\mathrm{s}} :: [\boldsymbol{a}]$ という型注釈を持つ．これは $x_{\mathrm{s}} :: []_{\boldsymbol{a}}$ のシンタックスシュガーである．[94]

型 $\boldsymbol{a}$ の Maybe の変数は $x_* :: \mathbf{Maybe}_{\boldsymbol{a}}$ という型注釈を持つ．

普段遣いの関数 $f :: \boldsymbol{a} \to \boldsymbol{a}$ をリスト変数 $x_{\mathrm{s}} :: [\boldsymbol{a}]$ に適用する場合は次のように書く．[95]

$$z_{\mathrm{s}} = f * x_{\mathrm{s}}\tag{131}$$

同じく関数 $f :: \boldsymbol{a} \to \boldsymbol{a}$ を Maybe 変数 $x_* :: \mathbf{Maybe}_{\boldsymbol{a}}$ に適用する場合は次のように書く．

[93] Haskell では一般マップ演算子 $(\circledast)$ は `fmap` である．ただしその実装は与えられず，対象とする型に応じて定義されるものとする．例えばリストに対しては `fmap = map` と定義されている．

[94] Haskell では `xs :: [] a` と書く．

[95] Haskell では `zs = f map xs` と書く．

$$z_? = f \circledast_? x_? \tag{132}$$

リストも Maybe も元の型 $\boldsymbol{a}$ から派生しており，関数適用のための特別な演算子を持つことになる．そこで，リストや Maybe は**関手**という型クラスに属する，型パラメタを伴う型であるとする．関手の型クラスを Functor で表す．関手型クラスの $\boldsymbol{a}$ 型の変数を次のように型注釈する．[96]

$$x_* :: \mathsf{Functor} \supset \boldsymbol{f} \Rightarrow \boldsymbol{f_a} \tag{133}$$

型 $[]_{\boldsymbol{a}}$ や型 $\mathbf{Maybe}_{\boldsymbol{a}}$ は型 $\boldsymbol{a}$ のところに **Int** や **Double** を代入すると具体的な型となる抽象的な型であった．今度は [] や **Maybe** のほうも $\boldsymbol{f}$ と抽象化するのである．この $\boldsymbol{f}$ は**型コンストラクタ**と呼ぶ．型コンストラクタには，具体的な引数，例えば **Int** や **Double** を与えると具体的な型になる．

型クラス Functor に属する型は $\circledast$ 演算子を必ず持つ．演算子 $\circledast$ は次の形を持つ．[97]

$$z_* = f \circledast x_* \tag{134}$$

演算子 $\circledast$ の型は次のとおりである．

$$(\circledast) :: \mathsf{Functor} \supset \boldsymbol{f} \Rightarrow (\boldsymbol{a} \to \boldsymbol{b}) \to \boldsymbol{f_a} \to \boldsymbol{f_b} \tag{135}$$

もし変数 x_* の型がリストであれば $\circledast = *$ であると解釈する．

1.8.2 関手則

関手すなわち Functor 型クラスに求められるのは，マップ演算子 $(\circledast)$ を持つことだけではない．関手のマップ演算子は，次のふたつの規則を満たす必要がある．

恒等射の存在

関手は恒等射 $(\mathrm{id} \circledast)$ ただし $\mathrm{id} \circledast x_* = x_*$ を持つ．

結合則

マップ演算子 $(\circledast)$ は結合則 $(g \circ f) \circledast x_* = (g \circledast) \circ (f \circledast) x_*$ を満たす．

このふたつをまとめて**関手則**と呼ぶ．

まず関数 id を定義しておくと，これは引数をそのまま返す関数で

$$\mathrm{id}\, x = x \tag{136}$$

である．引数 x は変数でも関数でも良いので，ラムダ式を使って

$$\begin{aligned} \mathrm{id} &= \lozenge \\ &= \backslash\, x \mapsto x \end{aligned} \tag{137}$$

と書いたほうがわかり良いかもしれない．

恒等射の存在 $\mathrm{id} \circledast x_* = x_*$ とは $\mathrm{id} \circledast x_* = \mathrm{id}\, x_*$ ということであるから，両辺の x_* を削除して

$$\mathrm{id} \circledast = \mathrm{id} \tag{138}$$

[96] Haskell では `xm :: Functor f => f a` と書く．
[97] Haskell では `zm = f <$> xm` と書く．

となる．一言で言うと，関数 id ⊛ は関数 id と同じで，引数をそのまま返す．この法則は任意の型クラス $\boldsymbol{f}$ について Functor ⊃ $\boldsymbol{f}$ であるならば，任意の型 $\boldsymbol{f_a}$ に対して (id ⊛) ::…

📍 **TK** 関手則の説明

1.8.3 アプリカティブ関手

📍 **TK** アプリカティブ関手

マップ演算子をさらに汎用性のあるものにするために新しく考え出された演算子が**アプリカティブマップ演算子**である．

📍 **TK**

$$[f, g, h] \boxast [x, y, z] = [fx, fy, fz, gx, gy, gz, hx, hy, hz] \tag{139}$$

リストのアプリカティブマップ演算子 ⊠ の型は $[\boldsymbol{a} \to \boldsymbol{b}] \to [\boldsymbol{a}] \to [\boldsymbol{b}]$ である．

これは

$$\boxast :: \underbrace{[\boldsymbol{a} \to \boldsymbol{b}]}_{[f,g,h]} \to \underbrace{[\boldsymbol{a}]}_{[x_0,x_1,\ldots]} \to \underbrace{[\boldsymbol{b}]}_{[fx_0,fx_1,\ldots]} \tag{140}$$

と解釈すれば良い．

リストバージョンのアプリカティブマップ演算子 (⊠) の特別な場合として，左引数のリストの要素数が 1 の場合を考えると

$$[f] \boxast [x, y, z] = [fx, fy, fz] \tag{141}$$

であり，通常のマップ演算子 $(*)$ を使ったマップすなわち

$$[f] * [x, y, z] = [fx, fy, fz] \tag{142}$$

と右辺が一致する．つまり，マップ演算子はアプリカティブマップ演算子の特別な場合と考えることができる．実際，リストマップ演算子はアプリカティブマップ演算子から

$$f * x_{\mathrm{s}} = [f] \boxast x_{\mathrm{s}} \tag{143}$$

と定義できる．

Maybe バージョンについても考えてみよう．Maybe に包まれた関数 $f_?$ を Maybe な変数 $x_?$ にマップするアプリカティブマップ演算子 $\boxast_?$ を

$$f_? \boxast x_? = \texttt{case}\ x_{\mathrm{s}}\ \texttt{of} \begin{cases} \mathrm{Just}_x \dashrightarrow x \circledast_? x_? \\ \square \dashrightarrow \mathrm{Nothing} \end{cases} \tag{144}$$

で定義する．この Maybe バージョンのアプリカティブマップ演算子 $(\boxast_?)$ から Maybe バージョンのマップ演算子 $(\circledast_?)$ は

$$f \circledast_? x_? = \mathrm{Just}_f \boxast_? x_? \tag{145}$$

のように導出できる．

これらの関係を一般化して

$$f \boxast x_* = \langle f \rangle \circledast x_* \tag{146}$$

となるような**一般アプリカティブマップ演算子** ($\boxast$) を考える．ここに f は関数，x_* はリストや Maybe といったコンテナ型の変数すなわち**コンテナ変数**である．一般アプリカティブマップ演算子 ($\boxast$) から一般マップ演算子 ($*$) を導き出すには，式〜〜〜のように値コンストラクタが必要である．この一般化された値コンストラクタを**ピュア演算子**と呼ぶ．アプリカティブマップ演算子とピュア演算子を持つ型クラスを**アプリカティブ関手**と呼び，Applicative 型クラスと定義する．[98]

ピュア演算子をピュア値コンストラクタと呼ばないのは，単純に「ピュア値」というものがないからである．Functor 型クラスはリスト型や Maybe 型を抽象化したものであって，直接変数を生成できない．型クラスは，C++の用語で言えば純粋仮想クラスのようなものであるし，Objective-C の用語で言えばメタクラスであるからである．もちろんリストのピュア演算子は $[x]$ であるし，Maybe のピュア演算子は $\mathbf{Maybe}_x$ であり，それぞれ具体的な変数を生成する．しかし変数 x にピュア演算子を適用した $\langle x \rangle$ は抽象的な概念であり，そのような変数は実在しない．[99]

一般アプリカティブマップ演算子 ($\boxast$) は多様性によってそれぞれリストバージョンのアプリカティブマップ演算子 (...) や Maybe バージョンのアプリカティブマップ演算子 ($\boxast_?$) にオーバーライドされ，それぞれリスト値コンストラクタ ($[x]$), Maybe 値コンストラクタ ($\mathbf{Maybe}_x$) を用いることでリストバージョンのマップ演算子 ($*$), Maybe バージョンのマップ演算子 ($\circledast_?$) を生成することができる．リスト値コンストラクタ，Maybe 値コンストラクタはそれぞれピュア演算子 $\langle x \rangle$ をオーバーライドしたものであるから，結局，一般アプリカティブマップ演算子とピュア演算子のふたつがあれば，任意のクラスのマップ演算子を生成することができる．

アプリカティブマップ演算子，ピュア演算子に一般化されたバージョンがあるように，リストの $\emptyset$ や Maybe の Nothing を一般化した値が必要である．それを $\varnothing$ とする．$\varnothing$ には特段名前が無いので，本書では単に「空」と呼ぶことにしよう．

この節の最後に**アプリカティブスタイル**という記法を紹介しておこう．アプリカティブマップ演算子は連続して

$$\langle f \rangle \boxast x_* \boxast y_* \tag{147}$$

このようにアプリカティブマップ演算子を並べる書き方をアプリカティブスタイルと呼ぶ．

📍 TK マージ

演算子 $\circledast$ は関手型クラスの型の値に 1 引数関数を適用することを可能にした．一方で 2 引数関数を適用するのは若干面倒である．いま関数 f が 2 引数をとるとし，関手型クラスの型の変数 x_* と y_* があるとする．関数 f に変数 x_* を部分適用して関数 $f' = f \circledast x_*$ を作ると，定義によって関数 f' は関手型クラスの型の変数になる．そこで，関手型クラスの型の関数を関手型クラスの型の変数に適用する新しい演算子が必要になる．このような演算子を**アプリカティブマップ演算子**と呼び $\boxast$ で表す．アプリカティブマップ演算子を用いると 2 引数の関数適用は次のように書ける．

$$\begin{aligned} z_* &= f' \boxast y_* \\ &= f \circledast x_* \boxast y_* \end{aligned} \tag{148}$$

[98] Haskell では一般アプリカティブマップ演算子を <*> と書く．
[99] Haskell は一般のピュア演算子の実装を与えていない．変数の型に応じて対応する関数が適用される．

任意の変数または関数を関手型クラスの型に入れる**ピュア演算子**があり，次のように書く．[100]

$$z_* = \langle x \rangle \tag{149}$$

なおピュア演算子の名称は「純粋(pure)」であるが，意味合いはむしろ「不純(impure)」のほうが近い．

ピュア演算子を用いると，式 148 は次のように書ける．[101]

$$z_* = \langle f \rangle \boxast x_* \boxast y_* \tag{150}$$

式 150 はかつて次のように書くことが提案されたが，却下された．[102]

$$z_* = [\![f x_* y_*]\!] \ldots \textbf{採用されなかった文法} \tag{151}$$

ピュア演算子とアプリカティブマップ演算子を必ず持つ関手のことを**アプリカティブ関手**と呼び Applicative で表す．

いま関数 $f :: \boldsymbol{a} \to \boldsymbol{b}$ に対して，新たな関数 f_* ただし $f_* = \langle f \rangle$ を作ったとすると，関数 f_* は次の型を持つ．

$$f_* :: \mathsf{Applicative} \supset \boldsymbol{f} \Rightarrow \boldsymbol{f_{a \to b}} \tag{152}$$

アプリカティブマップ演算子は変数 $x_* :: \mathsf{Applicative} \supset \boldsymbol{f} \Rightarrow \boldsymbol{f_a}$ に対して，関数 f_* を $z_* = f_* \boxast x_*$ のように作用させる．変数 z_* の型は $z_* :: \mathsf{Applicative} \supset \boldsymbol{f} \Rightarrow \boldsymbol{f_b}$ である．

📍 **TK** マージ

恒等射の存在

アプリカティブ関手は恒等射 id ただし $\langle \mathrm{id} \rangle \boxast x_* = x_*$ を持つ．

Homomorphism

$\langle f \rangle \boxast \langle x \rangle = \langle f x \rangle$

Interchange

$\langle f \rangle \boxast \langle x \rangle = \langle \lozenge \$ x \rangle \boxast f$

Composition

$h_* \boxast (g_* \boxast f_*) = \langle \lozenge \circ \lozenge \rangle \boxast h_* \boxast g_* \boxast f_*$

1.8.4 余談：関数と関手

📍 **TK** 関数と関手

Function laws.

[100] Haskell では `z = pure x` と書く．
[101] Haskell では `zm = (pure f) <*> xm <*> ym` と書く．
[102] 現在の Haskell では `z = liftA2 f x y` と書くことで代用されている．元の提案は `z = [|f x y|]` であった．

$$\begin{aligned} f \circ \mathrm{id} = \mathrm{id} \circ f = f \\ (h \circ g) \circ f = h \circ (g \circ f) \end{aligned} \tag{153}$$

Functor laws.

$$\begin{aligned} \mathrm{id} \circledast = \mathrm{id} \\ (g \circ f) \circledast = (g \circledast) \circ (f \circledast) \end{aligned} \tag{154}$$

📍 **TK** リライト

関数は関手である．関手とはマップ演算子を持つ型クラスのことであった．そこで，関数がどのようなマップ演算子を持つのか考えてみる．

いま関数 f が $f :: \boldsymbol{r} \to \boldsymbol{q}$ という型を持っているとする．この式は $\to$ を二項演算子，すなわち 2 引数関数とみなせば $f :: (\to)\boldsymbol{rq}$ と等価である．全く形式的に，$((\to)\boldsymbol{r})$ なる型コンストラクタがあるとして

$$\boldsymbol{r} \to \boldsymbol{q} = ((\to)\boldsymbol{r})_{\boldsymbol{q}} \tag{155}$$

であると考えてみる．型 $\boldsymbol{q}$ から型コンストラクタ $((\to)\boldsymbol{r})$ によって型 $(\boldsymbol{r} \to \boldsymbol{q})$ が作られると考えるのだ．[103]

マップ演算子の型.

関数のマップ演算子 $\left(\circledast_{((\to)\boldsymbol{r})}\right)$ と関数の合成演算子 $(\circ)$ は同じ型を持つ.

幸い，我々は関数のマップ演算子の実装に関しては，型さえ守っていれば（そして第〜〜〜章で述べる**関手則**さえ守っていれば）自由に選べる．そこで

$$\circledast_{((\to)\boldsymbol{r})} = \circ \tag{156}$$

としておこう．これが Haskell における関数のマップ演算子の定義である．

♣

関数はアプリカティブ関手でもある．アプリカティブ関手には，アプリカティブマップ演算子とピュア演算子が定義されるのであった．そこで，関数版のアプリカティブマップ演算子を $\boxast_{((\to)\boldsymbol{r})}$ とし，関数版のピュア演算子を $\langle x \rangle_{((\to)\boldsymbol{r})}$ と書くことにしよう．

ピュア演算子は $\boldsymbol{q} \to (\boldsymbol{r} \to \boldsymbol{q})$ 型を持たなければならない．従って関数版のピュア演算子は変数から関数を作るとも考えられる．我々は関数版のピュア演算子として

$$\langle x \rangle_{((\to)\boldsymbol{r})} = \backslash \square \mapsto x \tag{157}$$

を採用する．[104]

関数版のアプリカティブマップ演算子を $\boxast_{((\to)\boldsymbol{r})}$ とすると，その型は

$$\boxast_{((\to)\boldsymbol{r})} :: ((\to)\boldsymbol{r})_{\boldsymbol{a} \to \boldsymbol{b}} \to ((\to)\boldsymbol{r})_{\boldsymbol{a}} \to ((\to)\boldsymbol{r})_{\boldsymbol{b}} \tag{158}$$

つまり

$$\boxast_{((\to)\boldsymbol{r})} = (\boldsymbol{r} \to \boldsymbol{a} \to \boldsymbol{b}) \to (\boldsymbol{r} \to \boldsymbol{a}) \to (\boldsymbol{r} \to \boldsymbol{b}) \tag{159}$$

である．

[103] Haskell では $((\to)\boldsymbol{r})$ を `((->)r)` と書く．
[104] Haskell では $\langle x \rangle_{((\to)\boldsymbol{r})}$ を `const x` と書く．

我々は関数版アプリカティブマップ演算子として

$$g \boxast_{((\to)\boldsymbol{r})} f = \backslash\, x \mapsto gx(fx) \tag{160}$$

とする．これは，関数版のピュア演算子の定義と，一般マップ演算子と一般アプリカティブマップ演算子の関係 $f \circledast x_* = \langle f \rangle \boxast x_*$ から導かれる．すなわち

$$\begin{aligned} \langle g \rangle \boxast f &= \backslash\, x \mapsto \langle g \rangle x(fx) \\ &= \backslash\, x \mapsto (\backslash\, \square \mapsto g)x(fx) \\ &= \backslash\, x \mapsto g(fx) \\ &= g \circ f \end{aligned} \tag{161}$$

であるからである．

📍 **TK** マージ

$$f :: \boldsymbol{r} \to \boldsymbol{q} \tag{162}$$

$$f :: \langle \boldsymbol{r} \to \blacklozenge \rangle_{\boldsymbol{q}} \tag{163}$$

$$f :: \langle (\to)\boldsymbol{r}\blacklozenge \rangle_{\boldsymbol{q}} \tag{164}$$

```
f :: ((->) r) q
```

$$f_2 \circ f_1 = f_2 \circledast f_1 \tag{165}$$

$$\begin{gathered} \mathrm{id} \circ f = \mathrm{id} \circ f = f \\ (h \circ g) \circ f = h \circ (g \circ f) \end{gathered} \tag{166}$$

1.8.5 この章のまとめ

📍 **TK** この章のまとめ．

1.9 モナド

1.9.1 バインド演算子

📍 **TK** バインド演算子

一般マップ演算子をピュア演算子と一般アプリカティブマップ演算子に分解することで，式の見通しを良くすることができるアプリカティブスタイルという記法を採用できた．アプリカティブスタイルでは

$$f \circledast x_* \boxast y_* \boxast z_* \tag{167}$$

という風にコンテナ変数 x_*, y_*, z_* に関数 f を適用させることができる．コンテナ変数 x_*, y_*, z_* のいずれかが $\oslash$ であれば式全体の値が $\oslash$ になる．これは 3 個の計算を並列に行って，その結果をそれぞれ x'_*, y'_*, z'_* に入れておき，最後に関数 f に投げるという**計算構造**を具現化したものである．（関数 f は C で言えば `main` 関数に相当するであろう．）

しかしながら，アプリカティブスタイルでは変数に文脈を与えるタイミングがコンテナ変数を作るときのそれぞれ 1 回に限られている．そこで，任意のタイミングで変数に文脈を与えられるように，別な方法で一般マップ演算子を分解してみよう．

Maybe の例を思い出そう．Maybe 型の変数 $x_?$ はラップされた値 Just_x を持つのか，エラーを表す Nothing を持つのかを選べる．そこで，引数 x をとり何らかの計算をする関数 φ を考えよう．この関数 φ は引数 x の値次第ではエラーを表す Nothing を返す．例えば

$$\begin{aligned} gx \mid_{x \neq 0} &= \mathrm{Just}_x \\ \mid_{\mathrm{otherwise}} &= \mathrm{Nothing} \end{aligned} \tag{168}$$

といった関数が考えられる．変数 x は文脈を持っていないが，関数 φ を適用した結果である φx は文脈を持っていることに注意しよう．いま φx は Maybe という文脈を持っているから，我々は

$$z_? = \varphi x \tag{169}$$

という風に結果を Maybe 変数に保存しなければならない．今まで見てきた $y = fx$ や $y_* = f \circledast x_*$ の関係とは異なることに注意しよう．

1.9.2 モナド則

📍 **TK** モナド則

$$\begin{aligned} \mu \heartsuit \langle x \rangle &= \mu x \\ \langle \diamond \rangle \heartsuit x_* &= x_* \\ \nu \heartsuit \mu \heartsuit x_* &= \nu \heartsuit (\mu \heartsuit x_*) \end{aligned} \tag{170}$$

$(\mathbb{M}, \heartsuit, \langle \diamond \rangle)$ はモノイドである．

関数 μ に作用する**クライスリスター**演算子 $\star$ を $\mu^\star = (\mu \heartsuit \diamond)$ と定義する．クライスリスターを用いると，モナド則は次のように書き直せる．

$$\begin{aligned} \mu^\star \langle x \rangle &= \mu x \\ \langle \diamond \rangle^\star x_* &= x_* \\ (\nu^\star \mu)^\star x_* &= \nu^\star (\mu^\star x_*) \end{aligned} \tag{171}$$

Right Identity

$\varphi \heartsuit \langle x \rangle = \varphi x$

♡

Left Identity

$\langle \diamond \rangle \heartsuit x_* = x_*$

♡

Associativity

$\varphi \heartsuit (\psi \heartsuit x_*) = (\backslash\, y \mapsto \varphi \heartsuit (\psi y)) \heartsuit x_*$

♡

1.9.3 モノイド

📍 **TK**

整数全てからなる**集合**を $\mathbb{Z}$ で表すことにする．計算機科学で整数と言うと，本当の整数と，例えば -2^{63} から $2^{63} - 1$ までの間の整数の意味と両方あるが，今は前者の意味である．

集合 $\mathbb{Z}$ の任意の**元**（**要素**）z を $z \in \mathbb{Z}$ と書く．

二つの整数 $z_1, z_2 \in \mathbb{Z}$ があるとしよう．両者の間には**足し算** $(+)$ が定義されており，その結果すなわち**和**もまた整数である．ここで $z_1 + z_2 \in \mathbb{Z}$ であるとき，演算子 $+$ が集合 $\mathbb{Z}$ に対して**全域性**を持つと言う．

一般に集合 $\mathbb{A}$ の元に対して二項演算子 $\circledcirc$ が定義されていて，$a_1, a_2 \in \mathbb{A}$ のときに

$$a_1 \circledcirc a_2 \in \mathbb{A} \tag{172}$$

である場合，つまり演算子 $\circledcirc$ が集合 $\mathbb{A}$ に対して全域性を持つ場合，組み合わせ $(\mathbb{A}, \circledcirc)$ を**マグマ**と呼ぶ．組み合わせ $(\mathbb{Z}, +)$ はマグマの例であり，$(\mathbb{Z}, \times)$ もマグマの例である．

他に**論理集合** $\mathbb{B} = \{\top, \bot\}$ に対して，**論理和** $(\vee)$ は全域性を持つから，組み合わせ $(\mathbb{B}, \vee)$ はマグマであるし，同様に**論理積** $(\wedge)$ も全域性を持つから，組み合わせ $(\mathbb{B}, \wedge)$ もマグマである．論理集合 $\mathbb{B}$ とは論理型 **Bool** を数学風に言い換えたものである．[105]

マグマのうち，演算を 2 回続ける場合，その順序によって結果が異ならない，つまり

$$a_1 \circledcirc (a_2 \circledcirc a_3) = (a_1 \circledcirc a_2) \circledcirc a_3 \tag{173}$$

ただし $a_1, a_2, a_3 \in \mathbb{A}$ のとき，組み合わせ $(\mathbb{A}, \circledcirc)$ のことを**半群**と呼ぶ．この〜〜〜で表される性質を**結合性**と呼ぶ．組み合わせ $(\mathbb{Z}, +), (\mathbb{Z}, \times), (\mathbb{B}, \vee), (\mathbb{B}, \wedge)$ はすべて半群である．

ところで，整数全体の集合 $\mathbb{Z}$ には特別な元 $0 \in \mathbb{Z}$ がある．この元 0 は $z \in \mathbb{Z}$ のとき

$$0 + z = z + 0 = z \tag{174}$$

という性質を持つ．この 0 を演算 $+$ における**単位元**と呼ぶ．足し算のことを**加法**とも言うので 0 のことは**加法単位元**と呼ぶこともあるし，文字通り**零元**と呼ぶこともある．

一般に，$a \in \mathbb{A}$ として

$$0 \circledcirc a = a \circledcirc 0 = a \tag{175}$$

であるとき，元 0 を単位元と呼ぶ．[106]

組み合わせ $(\mathbb{A}, \circledcirc)$ が半群のとき，集合 $\mathbb{A}$ の単位元 0 との組み合わせ $(\mathbb{A}, \circledcirc, 0)$ のことを**モノイド**または**単位的半群**と呼ぶ．例えば $(\mathbb{Z}, +, 0)$ はモノイドであるし，$(\mathbb{Z}, \times, 1), (\mathbb{B}, \vee, \bot), (\mathbb{B}, \wedge, \top)$ もモノイドである．

このように，数学者は数の性質を抽象化し，集合とその集合に対する演算というものの見方をよく行う．プログラミングの言葉で言えば，複数のクラスに共通のインタフェースを定義するようなものである．

型 $\boldsymbol{a}$ を任意の型としたとき，型 $\boldsymbol{a} \to \boldsymbol{a}$ の関数もまたモノイドとなる．まず「何もしない」関数 id を次のように定義する．

$$\mathrm{id} = \lozenge \tag{176}$$

これはもちろん $\mathrm{id} = \backslash\, x \mapsto x$ の意味で，引数をそのまま返す関数である．

我々は関数を合成できる．そこで $\boldsymbol{a} \to \boldsymbol{a}$ 型の関数 f と関数 id の合成を考えると $\mathrm{id} \circ f = f \circ \mathrm{id} = f$ であり，また関数 $f, g, h :: \boldsymbol{a} \to \boldsymbol{a}$ について $(h \circ g) \circ f = h \circ (g \circ f)$ でるから，組み合わせ $(\boldsymbol{a} \to \boldsymbol{a}, \circ, \mathrm{id})$ はモノイドであることがわかる．

📍 **TK**

[105] 記号 $\top$ は True を，記号 $\bot$ は False を抽象化したものである．記号 $\wedge$ は and と読み，記号 $\vee$ は or と読む．

[106] 厳密には $0_{\text{left}} \circledcirc a = a$ のとき 0_{left} を**左単位元**と呼び，$a \circledcirc 0_{\text{right}} = a$ のとき 0_{right} を**右単位元**と呼ぶが，本書では両者を区別せず単位元と呼ぶ．

$(\mathbb{Z}, +, 0)$ はモノイドである．

同じ型を持つ関数を集めた関数の集合 $\mathbb{F}$ があるとする．任意の関数 $f \in \mathbb{F}$ について $\mathrm{id}\, f = f$ なる関数 id があり，かつ任意の関数 $f, g, h \in \mathbb{F}$ に対して $(h \circ g) \circ f) = h \circ (g \circ f)$ が成り立つとする．このとき組み合わせ $(\mathbb{F}, \circ, \mathrm{id})$ は**モノイド**であるという．

- 単位元の存在
- 結合律

1.9.4 余談：IO サバイバルキット 3

1.9.5 この章のまとめ

1.10 IO

$$\begin{aligned} &\mathbf{main} :: \mathbf{IO}_{\mathbf{Int}} \\ &\mathbf{main} = \langle 0 \rangle \end{aligned} \tag{177}$$

1.10.1 IO

📍 **TK** IO

1.10.2 IO モナド

1.10.3 制御構造

1.10.4 余談：do 記法

📍 **TK**

```python
import sys
def read_and_write():
  s = readline()
  print(s)
read_and_write()
```

```haskell
-- Haskell
main = do { s <- getLine; print s }
```

```haskell
-- Haskell
main = do
  s <- getLine
  print s
```

$$\begin{gathered} \mathbf{main} = \mathtt{do}\ \{s \leftarrow \mathbf{getLine};\quad \mathbf{print}\ s\} \\ \Updownarrow \\ \mathbf{main} = \mathbf{getLine} \vec{\heartsuit} (\backslash\, s \mapsto \mathbf{print}\ s) \\ \Updownarrow \\ \mathbf{main} = \mathbf{print} \heartsuit \mathbf{getLine} \end{gathered} \tag{178}$$

```python
import sys
def read_and_write():
  s = readline()
  t = f(s)
```

```
  print(t)
read_and_write()
```

```
--Haskell
-- f :: String -> String
main = do { s <- getLine; let t = f s in print t}
```

```
--Haskell
-- f :: String -> String
main = do
  s <- getLine
  let t = f s
  print t
```

$$
\begin{gathered}
\mathbf{main} = \texttt{do}\ \{s \leftarrow \mathbf{getLine};\ \ \texttt{let}\ t \triangleq fs\ \texttt{in}\ \mathbf{print}\ t\} \\
\Updownarrow \\
\mathbf{main} = \mathbf{getLine} \vec{\heartsuit} (\backslash\, s \mapsto \texttt{let}\ t \triangleq fs\ \texttt{in}\ \mathbf{print}\ t) \\
\Updownarrow \\
\mathbf{main} = (\backslash\, s \mapsto \mathbf{print} \heartsuit fs) \heartsuit \mathbf{getLine}
\end{gathered}
\tag{179}
$$

1.10.5 この章のまとめ

2 Haskell プログラミング

3 プログラミングと代数構造

3.1 整数

3.2 条件式

3.3 Y コンビネータ

4 Haskell の深い部分

4.1 型と種

4.1.1 型の計算

4.1.2 種

📍 **TK** 種

$$
\mathfrak{K} \rightarrow \mathfrak{Type} \tag{180}
$$

See https://zenn.dev/mod_poppo/books/haskell-type-level-programming/viewer/types-and-kinds

4.2 マクロと Template Haskell

📍 **TK** マクロ

5 執筆ノート

項目	内容	Haskell コード
Function application	$z = fx$	`z = f x`
List map	$z_{\mathrm{s}} = f * x_{\mathrm{s}}$	`zs = f `map` xs`
Maybe map	$z_? = f \circledast_? x_?$	`zm = f <$> xm`
Functor map	$z_* = f \circledast x_*$	`zm = f <$> xm`
Applicative map	$z_* = f_* \boxast x_*$	`zm = fm <*> xm`
Applicative map with 2 arguments	$z_* = g_* \boxast x_* \boxast y_*$	`zm = gm <*> xm <*> ym`
Applicative map with 2 arguments	$z_* = g \circledast x_* \boxast y_*$ $= [\![g x_* y_*]\!]$	`zm = gm <$> xm <*> ym`
Monadic function application	$z_* = \varphi x$	`zm = phi x`
Bind	$z_* = \varphi \heartsuit x_*$	`zm = phi =<< xm`
Double bind（右結合）	$z_* = \psi \heartsuit \varphi \heartsuit x_*$	`zm = psi =<< phi =<< xm`

$$
\begin{aligned}
&\texttt{class}\ \mathsf{Eq} \supset \boldsymbol{a}\ \texttt{where} \\
&\qquad (\equiv) :: \boldsymbol{a} \to \boldsymbol{a} \to \mathbf{Bool} \\
&\qquad (\not\equiv) :: \boldsymbol{a} \to \boldsymbol{a} \to \mathbf{Bool} \\
&\qquad (\equiv) = \neg(x \not\equiv y) \\
&\qquad (\not\equiv) = \neg(x \equiv y)
\end{aligned} \tag{181}
$$

$$
\texttt{data}\ \mathrm{TrafficLight} = \mathrm{Red} \curlyvee \mathrm{Yellow} \curlyvee \mathrm{Green} \tag{182}
$$

$$
\begin{aligned}
&\texttt{instance}\ \mathsf{Eq} \supset \mathrm{TrafficLight}\ \texttt{where} \\
&\qquad \mathrm{Red} \equiv \mathrm{Red} = \mathrm{True} \\
&\qquad \mathrm{Yellow} \equiv \mathrm{Yellow} = \mathrm{True} \\
&\qquad \mathrm{Green} \equiv \mathrm{Green} = \mathrm{True} \\
&\qquad \square \equiv \square = \mathrm{False}
\end{aligned} \tag{183}
$$